

MySQL and the SQL Language

Student Workbook 7a – Databases

Version 7.1 Y

Table of Contents

Module 1 Introduction to Relational Databases	5
Section 1–1 Relational Databases	6
Databases and DBMS.....	7
Database Operations	8
Relational Databases	9
SQL	10
Databases: A Collection of Related Tables	11
Normalization: Minimizing Redundancy	13
Section 1–2 Working with MySQL	14
Installing MySQL Server and Understanding the Database	15
Working with MySQL Workbench	17
MySQL Navigator	18
Exploring a Database	19
Viewing Records	22
Executing Queries.....	23
Exercises – Northwind Traders.....	25
Module 2 The SQL Language and Querying Data	26
Section 2–1 Querying Data	27
The SQL Language.....	29
Using SELECT Statements to Query Data	30
Example: Simple Select.....	31
Example: <code>SELECT *</code>	32
Optional Clauses in a <code>SELECT</code> Statement.....	33
<code>ORDER BY</code> Clause	34
Examples: <code>ORDER BY</code>	35
<code>WHERE</code> Clause.....	38
Example: <code>WHERE</code>	39
Example: Complex <code>WHERE</code>	40
Using <code>BETWEEN</code> for Comparisons.....	41
Example: <code>BETWEEN</code>	42
Using <code>LIKE</code> for Comparisons	43
Example: <code>LIKE</code>	44
Querying for <code>NULL</code> Values	45
Example: Querying with <code>NULL</code>	46
<code>SELECT DISTINCT</code>	47
Example: <code>DISTINCT</code>	48
Comments.....	49
Exercises- Basic Queries	50
Module 3 Additional Querying Features	53
Section 3–1 Aggregate Functions	54
Aggregate Functions	55
Example: <code>COUNT ()</code>	56
Example: <code>SUM ()</code>	57
Example: <code>AVG ()</code>	58
Example: <code>MIN ()</code> and <code>MAX ()</code>	59
Section 3–2 Grouping Results.....	60
<code>GROUP BY</code> clause.....	61
Example: <code>GROUP BY</code>	62
Renaming Computed Fields - Using <code>AS</code>	64
Example: <code>AS</code> keyword.....	65
<code>HAVING</code> clause.....	66
Example: <code>HAVING</code>	67

Exercises – Aggregates and Grouping	68
Section 3–3 Nested Queries	69
Nested Queries	70
Example: Nested Query	71
Exercises – Nested Queries	73
Module 4 Querying Multiple Tables (JOINS)	74
Section 4–1 Querying Multiple Tables (JOINS)	75
Querying Multiple Tables (JOINS)	76
Inner Joins	77
Example: INNER JOIN	79
Beyond Inner Joins	81
Outer Joins	83
Example: Outer Joins	84
Viewing the Database	85
Exercises - Joins	86
Module 5 Modifying the Data and the Database	87
Section 5–1 Inserting, Updating and Deleting Data	88
Inserting, Updating and Deleting Data	89
INSERT INTO Statement	90
Example: INSERT INTO	91
UPDATE Statement	92
Example: UPDATE	93
DELETE Statement	94
Example: DELETE	95
Exercises – Insert, Update and Delete	96
Section 5–2 Creating a Table	97
Creating a New Table	98
Creating a New Table	99
Example: CREATE TABLE	100
Module 6 Connecting SQL to Java	101
Section 6–1 Maven Dependencies	102
The pom.xml file	103
Adding the MySQL Driver	104
Adding the DataSource Dependency	105
Section 6–2 Introduction to JDBC	106
Java Applications and Databases	107
What is JDBC	108
JDBC Architecture	109
The Database URL	110
Creating and Connection to a DataSource	111
The JDBC Programming Process	112
Try-With-Resources	113
Processing the ResultSet	115
Exercise – NorthwindTraders Pt. 1	116
Section 6–3 Working with Prepared Statements	117
What is a PreparedStatement?	118
Creating a PreparedStatement	119
Setting Parameter Values	120
Executing Queries	121
Exercise – NorthwindTraders Pt. 2	123
Exercise – NorthwindTraders Pt. 3	124
Section 6–4 SQL Injection Prevention	125
What is SQL Injection?	126
How SQL Injection Works	127

Why PreparedStatements are Safer	128
Exercise – NorthwindTraders Pt. 4.....	129
Module 7 Continued Learning	130
Other Features.....	131
Other SQL Learning Tools	132
Section 7–1 Practice Repository.....	133
Practice Problems.....	134
Using AI Tools for SQL Data Generation	135

Module 1

Introduction to Relational Databases

Section 1–1

Relational Databases

Databases and DBMS

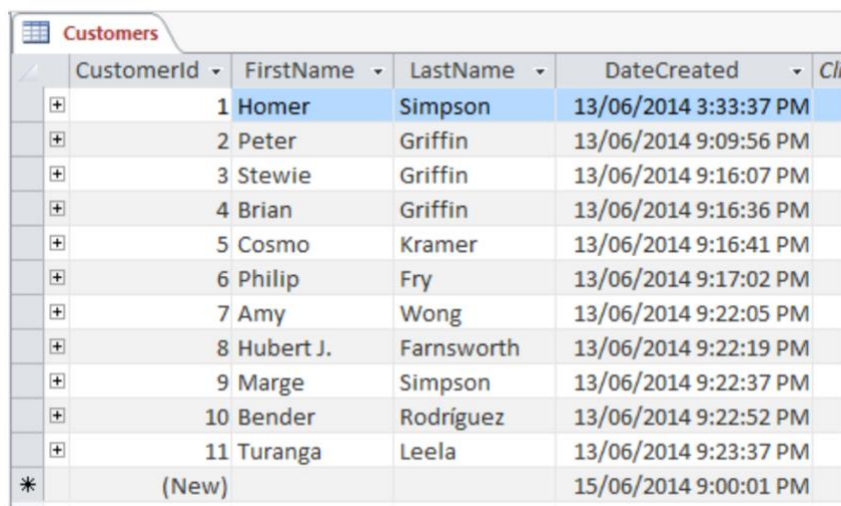
- **Databases are collections of organized data**
- **A database management system (DBMS) can interact with that data**
- **There are many DBMS systems out there, including:**
 - MySQL
 - SQL Server
 - Oracle
 - PostgreSQL
 - MongoDB
- **Modern DBMS fall into two broad categories:**
 - ***SQL based databases*** are typically relational databases and they manage their data in collections of related tables
 - **NoSQL databases** have entirely different ways of managing data compared to traditional SQL databases, with several common types:
 - * **Document databases** (ex: MongoDB) store data as JSON-like documents, making them flexible for varied data structures
 - * **Graph databases** (ex: Neo4j) store data as nodes and relationships, ideal for highly connected data like social networks
 - * **Key-value databases** (ex: Redis) store data as simple key-value pairs, providing extremely fast lookups for caching
 - * **Column-family databases** (ex: Cassandra) organize data in columns rather than rows, optimizing for massive scale

Database Operations

- **When we store data in a database, there are four basic types of operations we perform with data**
 - Query the data for answers to questions
 - Search and find data
 - Add new data
 - Change data
 - Delete data
- **There is a common acronym for these four types of operation: CRUD**
 - Create
 - Read
 - Update
 - Delete

Relational Databases

- With relational databases, the data is organized in around tables made up of rows and columns.
 - Each **column** represents a "property" we want to store
 - Each **row** represents one set of data and is often called a "record" or "entry" (or row!)



	CustomerId	FirstName	LastName	DateCreated	Client
+	1	Homer	Simpson	13/06/2014 3:33:37 PM	
+	2	Peter	Griffin	13/06/2014 9:09:56 PM	
+	3	Stewie	Griffin	13/06/2014 9:16:07 PM	
+	4	Brian	Griffin	13/06/2014 9:16:36 PM	
+	5	Cosmo	Kramer	13/06/2014 9:16:41 PM	
+	6	Philip	Fry	13/06/2014 9:17:02 PM	
+	7	Amy	Wong	13/06/2014 9:22:05 PM	
+	8	Hubert J.	Farnsworth	13/06/2014 9:22:19 PM	
+	9	Marge	Simpson	13/06/2014 9:22:37 PM	
+	10	Bender	Rodríguez	13/06/2014 9:22:52 PM	
+	11	Turanga	Leela	13/06/2014 9:23:37 PM	
*	(New)			15/06/2014 9:00:01 PM	

- When you create a table, you must specify:
 - the names of each column
 - the type of data (int, string, etc) held in each column
 - You can also set other properties for each column, such as whether it can be null

SQL

- **Modern relational database management systems use a language called SQL to interact with the data**
 - SQL stands for "Structured Query Language"
- **Programmers are opinionated about how to pronounce SQL**
 - The standard says that "Ess-cue-ell" is the correct way pronounce the language
 - Famous authors who originally wrote about the technology, including Jennifer Widom and Christopher Date, pronounce it "sequel"
- **A command to fetch data written using SQL is called a "query" and might look like this:**

Example

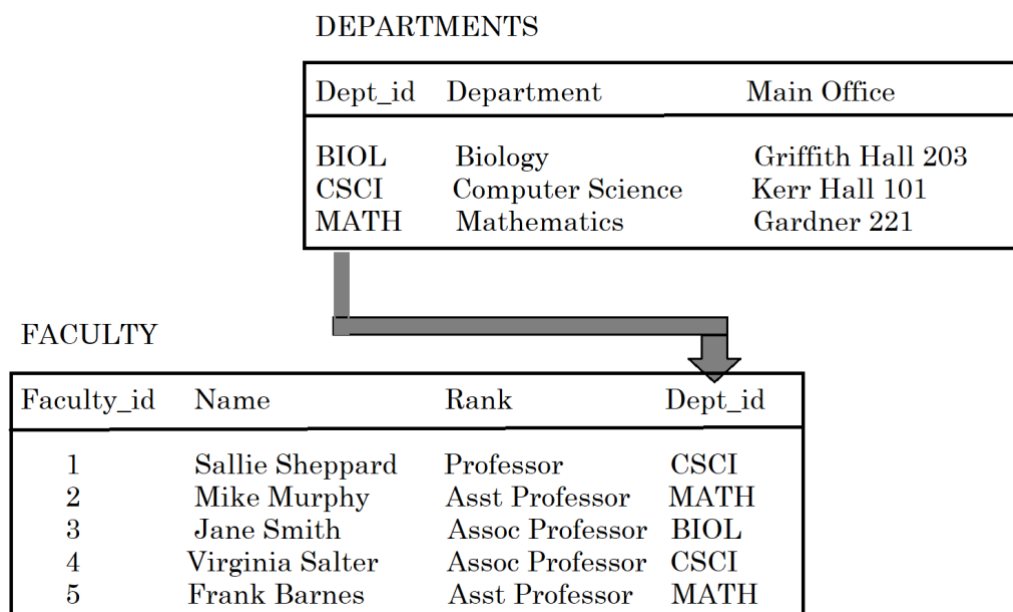
```
SELECT FirstName, LastName  
FROM Customers  
WHERE LastName = 'Griffin'
```

- **The information returned from a query is called the "result" or a "result set".**

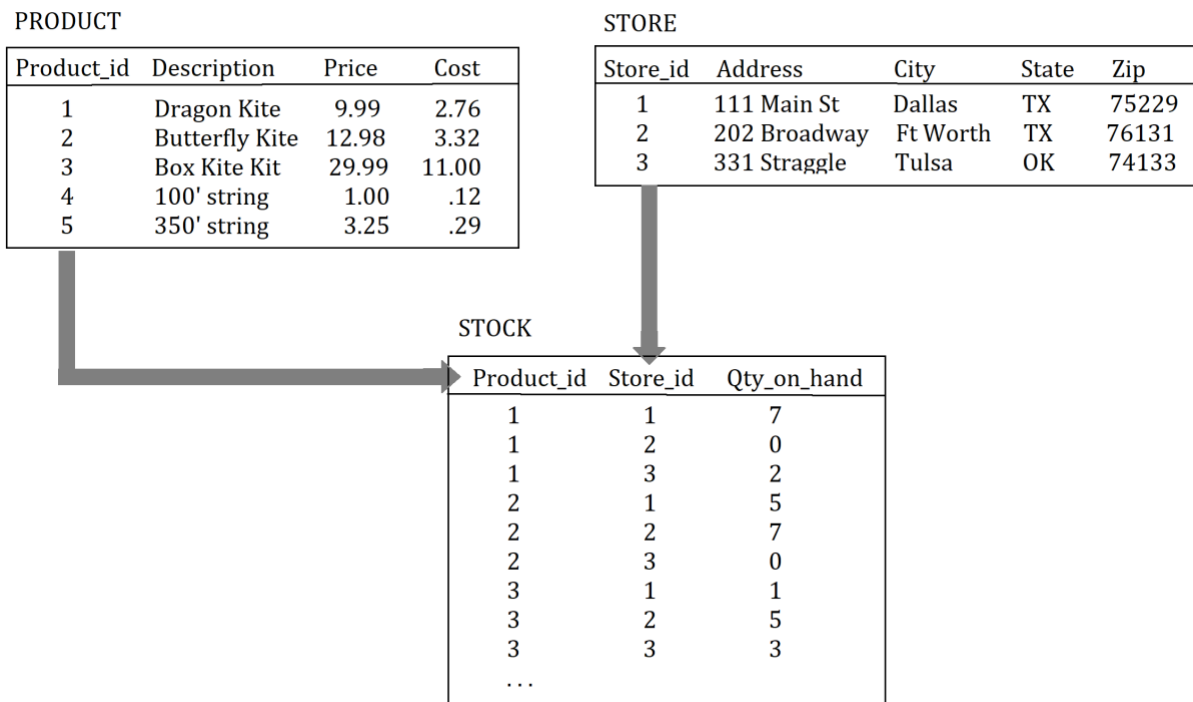
Peter	Griffin
Stewie	Griffin
Brian	Griffin

Databases: A Collection of Related Tables

- Relational databases typically consist of many tables
- Typically, each table contains one type of thing
 - ex: Products, stores, orders, etc
- Relationships between tables are established by having associated columns
 - The column that uniquely identifies a row is called the **primary key**
 - When a column in one table refers to the primary key of another table, it is called a **foreign key** and establishes a relationship between the two tables
- One-to-many relationships are represented as shown here:



- Many-to-many relationships are represented using a third table called an *intersection table*, as shown here:



Normalization: Minimizing Redundancy

- **Reducing redundancy is one of the goals of designing tables**
 - When you duplicate data, then changes to data become complicated because you have to make changes multiple places
 - The process of designing a database that reduces redundancy is called *normalization*
- **There's no "one answer" to database design**
 - It is a matter of give and take and what is right for the situation
 - For example, how do you represent Customers and Addresses?
- **One organization might include: customers and their addresses in a single table**

CustomerID	CompanyName	Address	City	Region	PostalCode	Country
ALFKI	Alfreds Futterkiste	Obere Str. 57	Berlin	NULL	12209	Germany
ANATR	Ana Trujillo Emparedados y helados	Avda. de la Constitucion 2222	Mxico D.F.	NULL	05021	Mexico
ANTON	Antonio Moreno Taquera	Mataderos 2312	Mxico D.F.	NULL	05023	Mexico
AROUT	Around the Horn	120 Hanover Sq.	London	NULL	WA1 1DP	UK
BERGS	Berglunds snabbkp	Berguvsvgen 8	Lule	NULL	S-958 22	Sweden
BLAUS	Blauer See Delikatessen	Forsterstr. 57	Mannheim	NULL	68306	Germany

- Another organization might be aware that multiple customers live at the same address and separate address info into one table and customer info into another
- A row in the Customer table is linked to a specific row in the Address table using a key

Address Table:

addressId	addressLine1	addressLine2	city	state	zip
1	222 SW Main Street		Tulsa	OK	55422
2	543 North Washington Avenue		Portland	OR	97204
3	998 Blair Blvd.	Suite 34	New York	NY	22001

Customer Table:

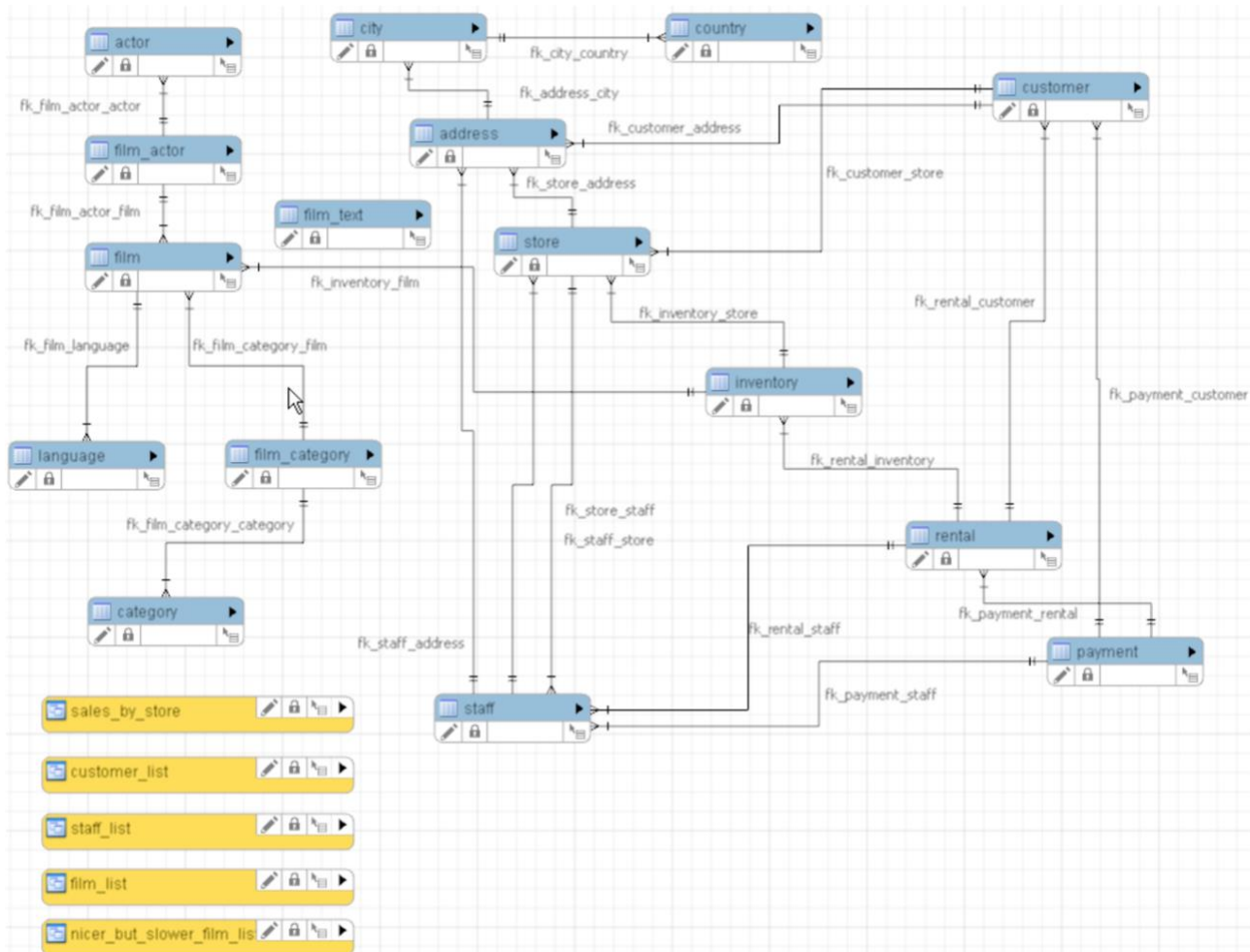
customerId	firstName	lastName	addressId
1	Jimmy	Stewart	1
2	Shelly	Duvall	3
3	Robert	Duvall	3
4	Fubar	Wilco	2

Section 1–2

Working with MySQL

Installing MySQL Server and Understanding the Database

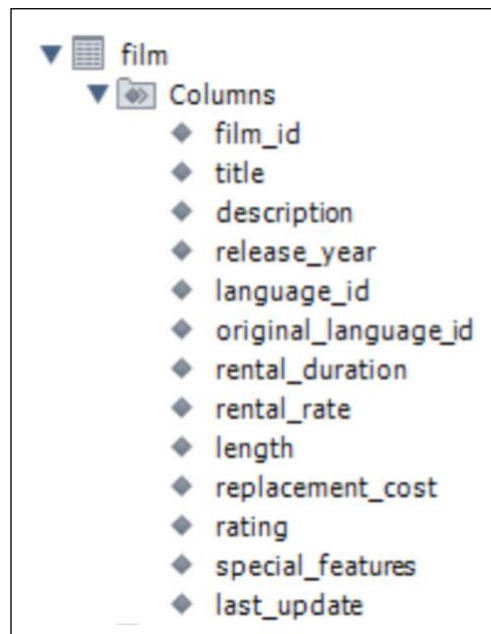
- The examples in this section of the course often use a sample database called sakila
- It is a database for a company that sells movies
 - You can read about the schema here:
<https://dev.mysql.com/doc/sakila/en/>



- **Many of our early queries will use the film table**

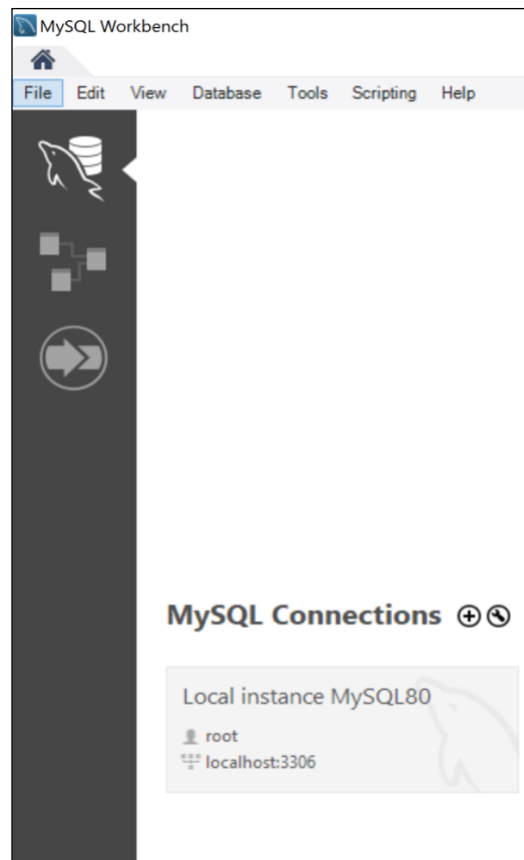
- The film table is a list of all films that potentially might in stock in the stores
- The actual in-stock copies of each film are represented in the inventory table.

- **The film table contains the following columns:**



Working with MySQL Workbench

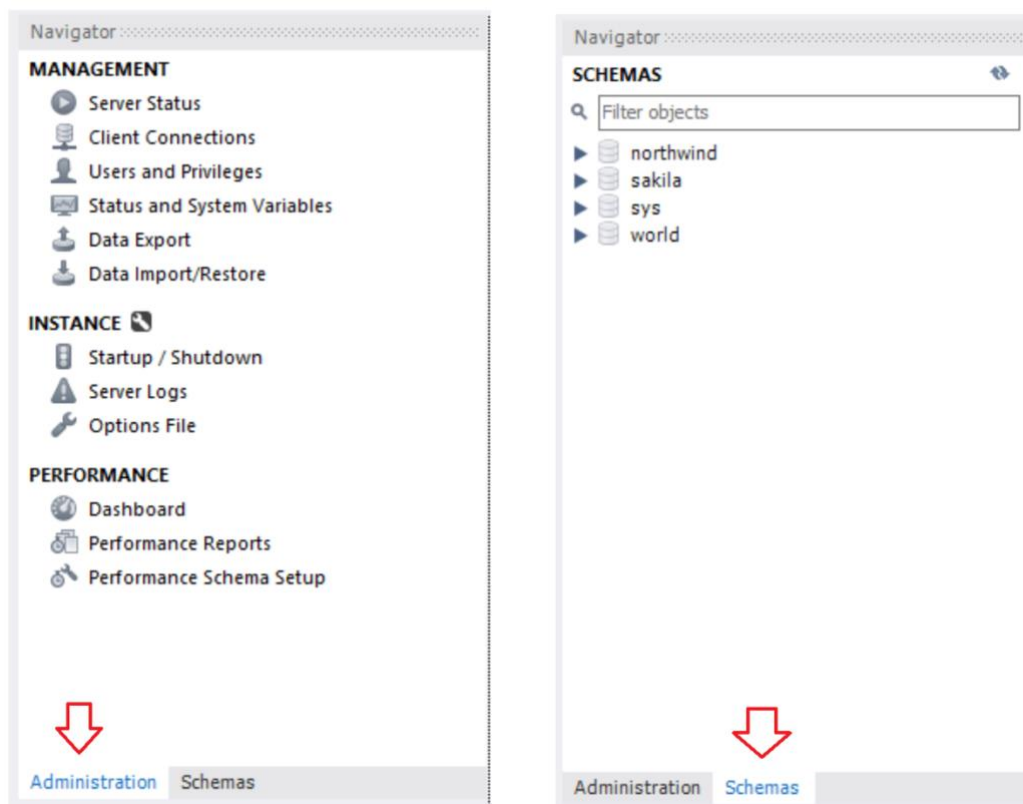
- You can run queries against MySQL databases using the MySQL Workbench
- Once you launch it, you should select your instance
 - Yours may look a little different, but should have your MySQL instance listed on the page



- When you select it, it will prompt you for your MySQL credentials

MySQL Navigator

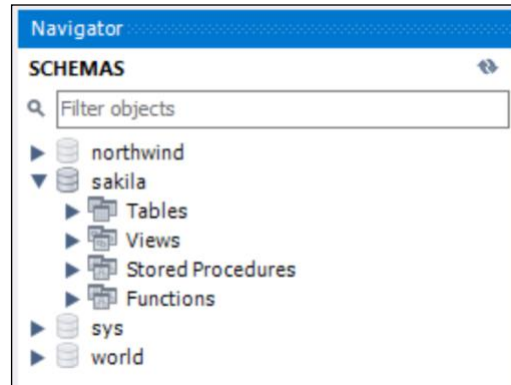
- The Navigator window docked on the left of the MySQL Workbench window lets you:
 - perform administrative tasks
 - work with your databases



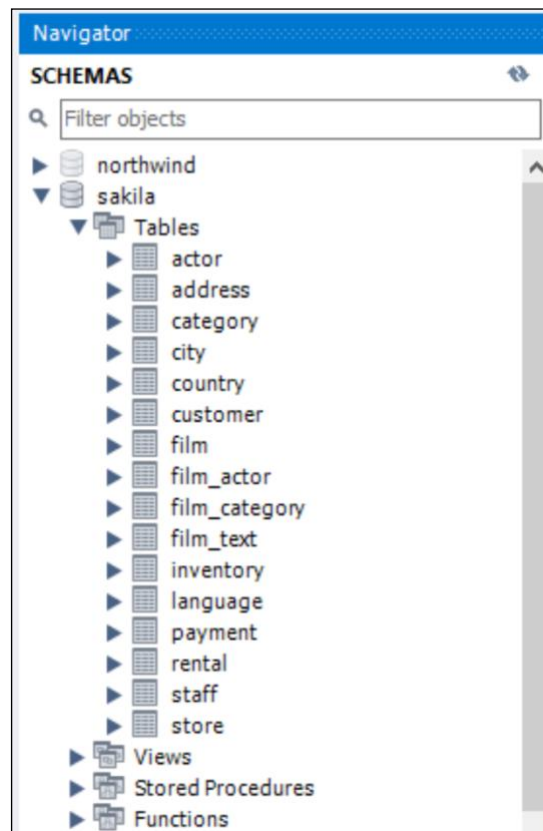
- When you switch to the schema tab, you will see your databases
 - You should have the Sakila and World databases installed as they are MySQL sample databases
 - You will have to explicitly add other databases you want to interact with

Exploring a Database

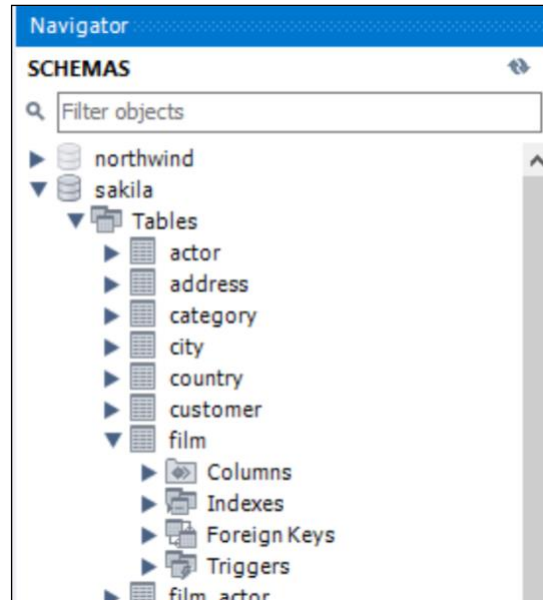
- If you drill into a database using Navigator, you can:
 - see the categories of elements in the database



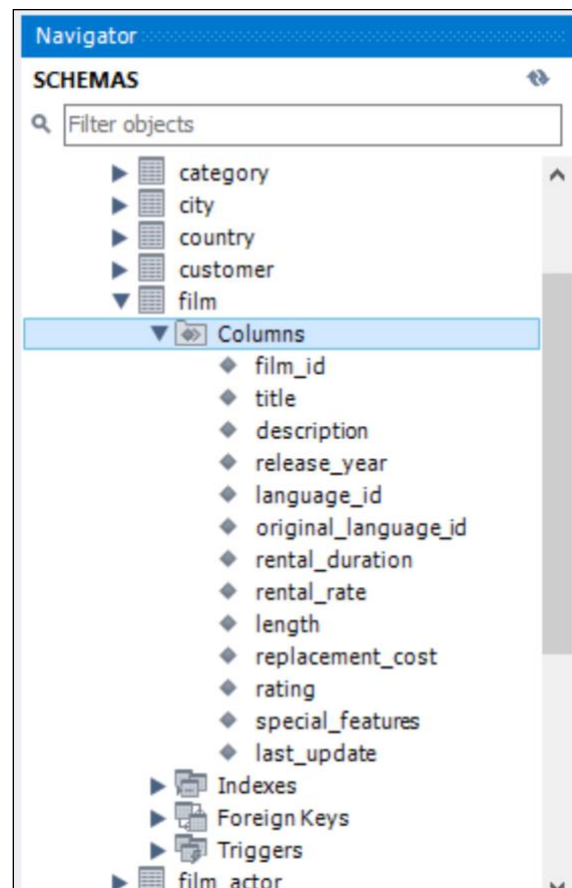
- see all the tables in the database



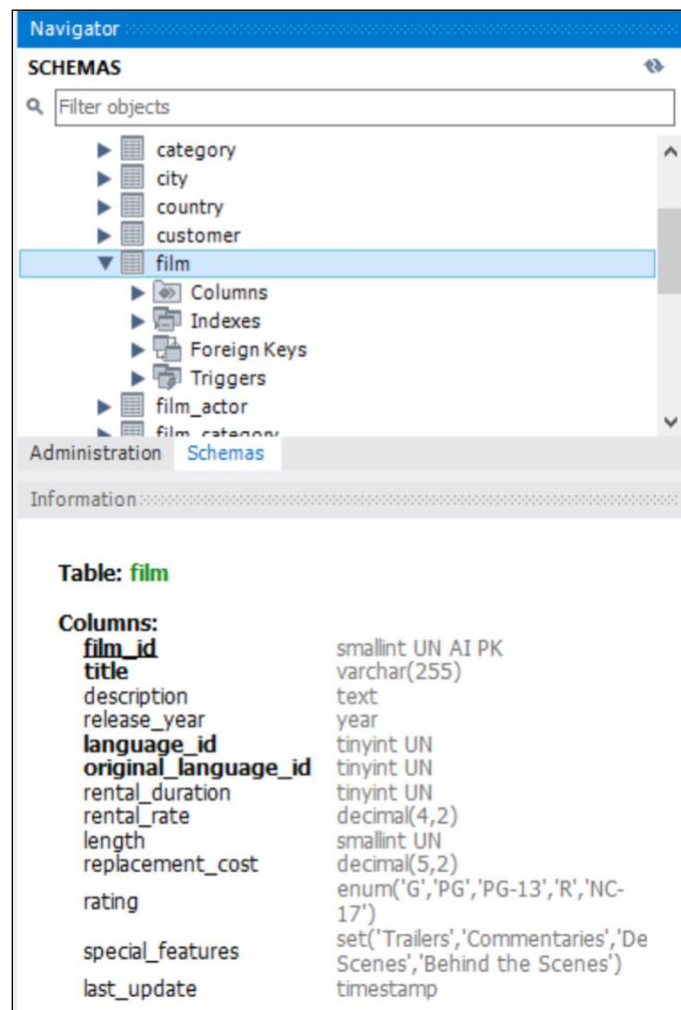
- see the characteristics of a table



- see the columns in a specific table

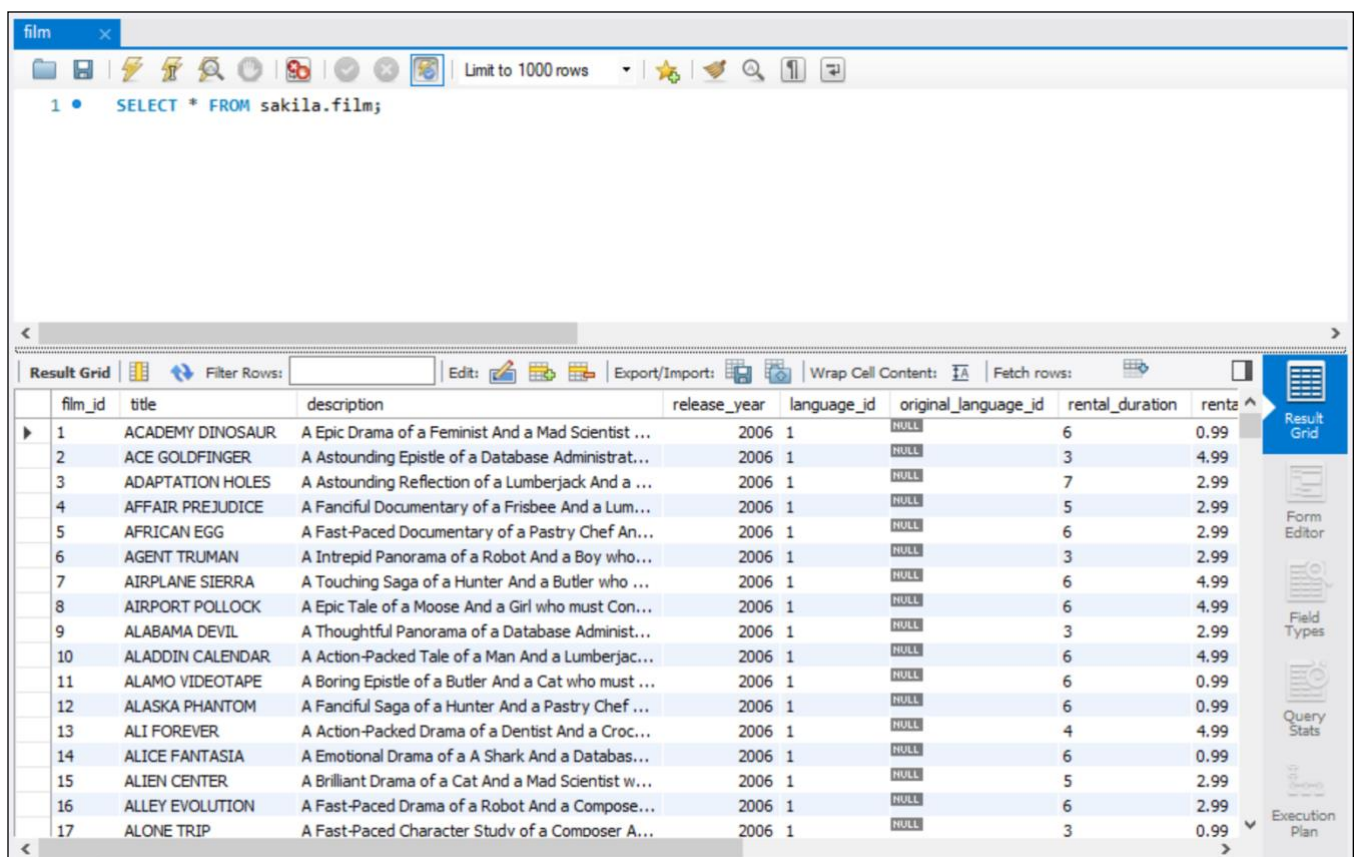


- see the type of each column in a table
- * when clicking on a table, look at the Information window



Viewing Records

- If you right-click on a table in the Navigator window, you can choose **Select Rows - Limit 1000** to view data
 - It opens the Query tab, generates a SELECT statement for the data, and runs the query



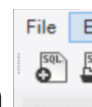
The screenshot shows a database application window titled "film". The top toolbar includes icons for file operations, a "Limit to 1000 rows" dropdown, and a "Result Grid" icon. The query editor displays the SQL statement: `1 • SELECT * FROM sakila.film;`

Below the query editor is the "Result Grid" tab, which displays a table of 17 records. The table has the following columns: `film_id`, `title`, `description`, `release_year`, `language_id`, `original_language_id`, `rental_duration`, and `rente`. The data is as follows:

film_id	title	description	release_year	language_id	original_language_id	rental_duration	rente
1	ACADEMY DINOSAUR	A Epic Drama of a Feminist And a Mad Scientist ...	2006	1	NULL	6	0.99
2	ACE GOLDFINGER	A Astounding Epistle of a Database Administrat...	2006	1	NULL	3	4.99
3	ADAPTATION HOLES	A Astounding Reflection of a Lumberjack And a ...	2006	1	NULL	7	2.99
4	AFFAIR PREJUDICE	A Fanciful Documentary of a Frisbee And a Lum...	2006	1	NULL	5	2.99
5	AFRICAN EGG	A Fast-Paced Documentary of a Pastry Chef An...	2006	1	NULL	6	2.99
6	AGENT TRUMAN	A Intrepid Panorama of a Robot And a Boy who...	2006	1	NULL	3	2.99
7	AIRPLANE SIERRA	A Touching Saga of a Hunter And a Butler who ...	2006	1	NULL	6	4.99
8	AIRPORT POLLOCK	A Epic Tale of a Moose And a Girl who must Con...	2006	1	NULL	6	4.99
9	ALABAMA DEVIL	A Thoughtful Panorama of a Database Administ...	2006	1	NULL	3	2.99
10	ALADDIN CALENDAR	A Action-Packed Tale of a Man And a Lumberjac...	2006	1	NULL	6	4.99
11	ALAMO VIDEOTAPE	A Boring Epistle of a Butler And a Cat who must ...	2006	1	NULL	6	0.99
12	ALASKA PHANTOM	A Fanciful Saga of a Hunter And a Pastry Chef ...	2006	1	NULL	6	0.99
13	ALI FOREVER	A Action-Packed Drama of a Dentist And a Croc...	2006	1	NULL	4	4.99
14	ALICE FANTASIA	A Emotional Drama of a A Shark And a Databas...	2006	1	NULL	6	0.99
15	ALIEN CENTER	A Brilliant Drama of a Cat And a Mad Scientist w...	2006	1	NULL	5	2.99
16	ALLEY EVOLUTION	A Fast-Paced Drama of a Robot And a Compose...	2006	1	NULL	6	2.99
17	ALONE TRIP	A Fast-Paced Character Study of a Composer A...	2006	1	NULL	3	0.99

The right sidebar contains icons for "Result Grid", "Form Editor", "Field Types", "Query Stats", and "Execution Plan".

- To open additional Query tabs, click the toolbar icon

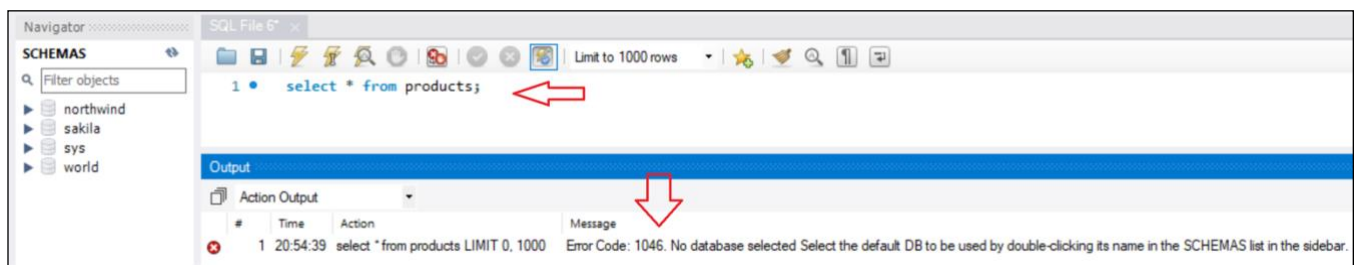


Executing Queries

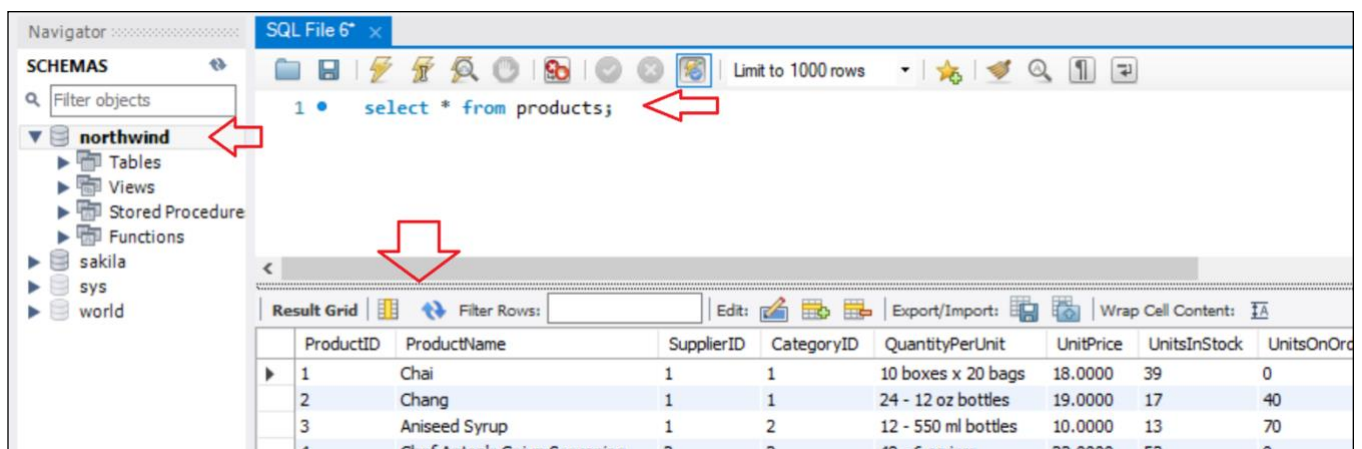
- You can enter any query you want in the Query tab and then execute it

– You execute it by specifying the  icon on the Query tab just above where you enter the SQL statement

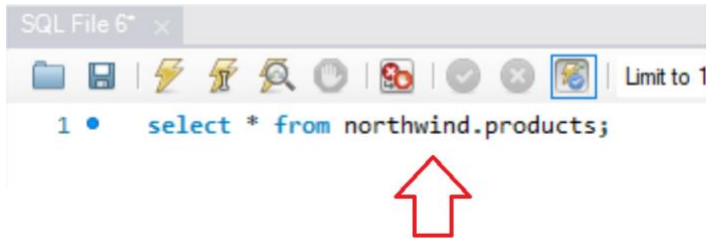
- However, because you may have several databases available, you have to select the database to run the query against



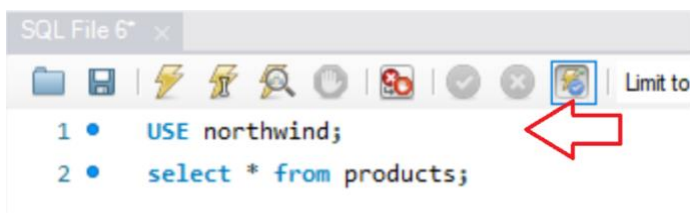
- One way to do this is to double-click the database schema in the Navigator
 - It will bold the database and become the default database that queries will be run against



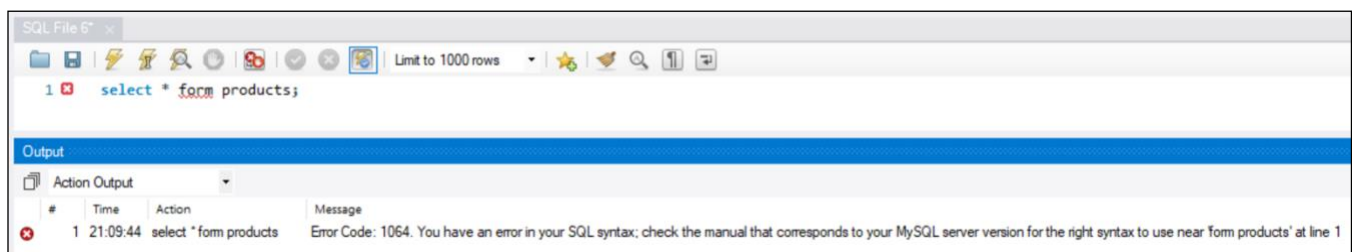
- Another way is to specify the database is to type it and a period in front of the table name



- Yet another way is to place a USE statement at the top of the script specifying the database



- If you have a syntax error in your query, the Output window will describe the error



- For more information on how MySQL Workbench works, see:

<https://dev.mysql.com/doc/workbench/en/wb-sql-editor-query-panel.html>

Exercises – Northwind Traders

EXERCISE 1

In this exercise, you will install the Northwind database and then run some simple queries against it.

We will use the Northwind database for many of the exercises and examples in this workbook. A script to install the database is included for you in the data files zip file.

Once you have installed your Northwind database, to see your new database in the Navigator window, you may have to refresh it. Right-click in the Navigator window and choose Refresh All.

Being able to use a database efficiently in your applications, requires that you to know your data. You should be familiar with the database structure and the data in the database.

Northwind is a database for a small grocery store. Take a few minutes to examine the schema. Then answer the following questions by looking at the data in the tables or the table structure.

NOTE: These question should all be answered **without** the use of SQL Queries

1. What categories of products does Northwind Traders sell?
2. What is the name of a product that sounds interesting to you?
3. What are some of the countries in which we have customers?
4. What date was our first order placed? How did you determine that?
5. What date was our last order placed?
6. How many customers do we currently have?
7. Do we have any suppliers in South America? Europe? Africa?
8. What is our most expensive product?
9. What is our least expensive product?
10. We have a customer named "The Big Cheese" if I wanted to get details about their last order from us how would you find that information?
 - a. What tables would you use?
 - b. How would you link the data in the tables you are using?
 - c. What information might I want to know about their order?

Module 2

The SQL Language and Querying Data

Section 2–1

Querying Data

The SQL Language

- **When you want to interact with SQL databases, you use SQL**
- **SQL is a *declarative* language**
 - Java and JavaScript are *imperative* languages
- **The difference is that, in JavaScript, you must specify *how* to do things that you want -**
 - Example: executing a group of statements multiple times with a `for` loop
- **With SQL, you specify *what* you want.**
 - All SQL databases have sophisticated execution optimizers that figure out the best way to produce your results based on what the database looks like when you execute your command
- **Over the next several modules, we will see examples of, and practice, SQL**

Using SELECT Statements to Query Data

- Much of the time, when we interact with SQL databases we will be looking for specific data to read
- In SQL, you do this with a SELECT statement
- The SELECT statement's minimum requirements are:
 - SELECT (a keyword)
 - the list of columns to be displayed (or * for all columns)
 - FROM (a keyword)
 - the table to get data from

Syntax

```
SELECT column1, column2, etc  
FROM table-name;
```

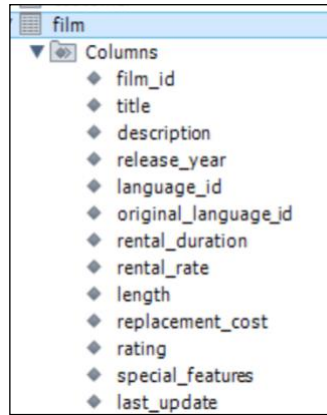
Syntax

```
SELECT *  
FROM table-name;
```

- The case of the keywords SELECT and FROM isn't important
 - It can be in uppercase, lowercase or mixed case
- The statement, and all other SQL statements, must end with a semicolon

Example: Simple Select

- **SAKILA DATABASE:** The film table is shown below:



A screenshot of a database interface showing the columns of the 'film' table. The table name 'film' is at the top. Below it, under the heading 'Columns', is a list of 13 columns: film_id, title, description, release_year, language_id, original_language_id, rental_duration, rental_rate, length, replacement_cost, rating, special_features, and last_update. Each column is preceded by a small diamond icon.

Columns
film_id
title
description
release_year
language_id
original_language_id
rental_duration
rental_rate
length
replacement_cost
rating
special_features
last_update

- **QUERY:** What are the titles of the films we sell and their ratings?
- **APPROACH:** Use a **SELECT** statement, list the columns we want after the keyword **SELECT**, and list the film table after the keyword **FROM**.

Query

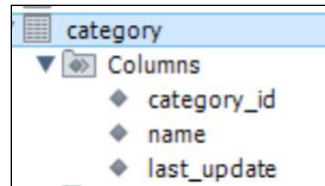
```
SELECT title, rating
FROM film
```

RESULTS:

```
+-----+-----+
| title                                | rating |
+-----+-----+
| ACADEMY DINOSAUR                    | PG     |
| ACE GOLDFINGER                      | G      |
+-----+-----+
| ... there are many rows returned ... |
+-----+-----+
```

Example: **SELECT** *

- **SAKILA DATABASE:** The `category` table is shown below:



category	
Columns	
category_id	
name	
last_update	

- **QUERY:** What are the categories of films we carry?
- **APPROACH:** Use a **SELECT** statement and list all columns from the `category` table.

Query

```
SELECT *  
FROM category
```

RESULTS:

```
+-----+-----+-----+  
| category_id | name          | last_update          |  
+-----+-----+-----+  
| 1           | Action        | 2006-02-15 04:46:27 |  
| 2           | Anumation     | 2006-02-15 04:46:27 |  
+-----+-----+-----+  
| ... there are many rows returned ... |  
+-----+-----+-----+
```

Optional Clauses in a `SELECT` Statement

- The `SELECT` statement has several optional clauses, including:
 - `ORDER BY` specifies how to order the returned rows
 - `WHERE` specifies which rows to retrieve
 - `AS` can be used to provide an alias for a column or expression in the `SELECT`
 - `GROUP BY` groups rows based on a specified characteristic so that an aggregate function can be applied to each group
 - `HAVING` specifies which groups to include in the result

ORDER BY Clause

- **ORDER BY** lets you control the order of the data that is returned
 - **ORDER BY** is generally the last clause in the **SELECT** statement
- By default, it returns data in ascending order based on the column selected

Syntax

```
SELECT column1, column2, etc  
FROM table  
ORDER BY any-column;
```

- However, you can use the **DESC** keyword after the column name to specify rows should be returned in descending order

Syntax

```
SELECT column1, column2, etc  
FROM table  
ORDER BY any-column DESC;
```

- **NOTE:** In some situations, you may want to have more than one field you order by
 - For example, order by state and, within state, by city

Examples: ORDER BY

- **QUERY:** What are the films we carry ordered by the length of the film (shortest first)?
- **APPROACH:** Use an **ORDER BY** on the query to sort by the length column.

Query

```
SELECT film_id, title, length
FROM film
ORDER BY length;
```

Results:

```
+-----+-----+-----+
| film_id | title           | length |
+-----+-----+-----+
| 15      | ALIEN CENTER    | 46     |
| 469     | IRON MOON       | 46     |
+-----+-----+-----+
| ... there are many rows returned ... |
+-----+-----+-----+
```

- **QUERY:** What are the films we carry ordered by the length of the film (longest first)?
- **APPROACH:** Use an **ORDER BY** on the query to sort by the length column and used a descending sort

Query

```
SELECT film_id, title, length
FROM film
ORDER BY length DESC;
```

RESULTS:

film_id	title	length
141	CHICAGO NORTH	185
182	CONTROL ANTHEM	185
... there are many rows returned ...		

- **QUERY:** What are the films we carry ordered by the length of the film (longest first), but if multiple films have the same length then order by **film_id**?
- **APPROACH:** Use an **ORDER BY** on the query to sort by the length column and used a descending sort

Query

```
SELECT film_id, title, length
FROM film
ORDER BY length DESC, film_id;
```

RESULTS:

film_id	title	length
141	CHICAGO NORTH	185
182	CONTROL ANTHEM	185
... there are many rows returned ...		

WHERE Clause

- The **WHERE** clause limits the rows returned to those that match a specified condition
- The condition is specified using the comparison operators:
 - = (equals) <> (not equals)
 - > >= < <= (comparisons)

Syntax

```
SELECT column1, column2, etc  
FROM table  
WHERE expression
```

- You can build a complex condition using the logical operators **AND**, **OR**, and **NOT**
- You can use parentheses to control order of precedence
- **NOTE:** String comparisons in MySQL are typically case insensitive

Example: WHERE

- **QUERY:** What films have a "PG" rating?
- **APPROACH:** Use a **WHERE** clause and specify a condition that the rating column must be "equal to" the string "PG".

Query

```
SELECT film_id, title, rating
FROM film
WHERE rating = "PG";
```

RESULTS:

```
+-----+-----+-----+
| film_id | title           | rating |
+-----+-----+-----+
| 1       | ACADEMY DINOSAUR | PG     |
| 6       | AGENT TRUMAN    | PG     |
+-----+-----+-----+
| ... there are many rows that match ... |
+-----+-----+-----+
```

Example: Complex WHERE

- **QUERY:** What films have a "PG" rating that run between 90 and 120 minutes? List the results in descending order by length. If two or more films have the same length, then list them in alphabetical order by title.
- **APPROACH:** Use a **WHERE** clause and specify a condition that the rating column must be "equal to" the string "PG" and that the value in the rating column must be between 90 and 120.

Query

```
SELECT film_id, title, rating, length
FROM film
WHERE rating = "PG" AND (length >= 90 AND length <= 120)
ORDER BY length DESC, title;
```

RESULTS:

film_id	title	rating	length
477	JAWVREAKER BROOKLYN	PG	118
645	OTHERS SOUP	PG	118
... there are many rows that match ...			

Using BETWEEN for Comparisons

- You can use the **BETWEEN** operator in a **WHERE** clause to specify a range of values
 - Specify the minimum and maximum values separated by the word **AND**
 - * NOTE: The values are inclusive in the range.

Example

```
WHERE length BETWEEN 89 AND 91
```

Example: BETWEEN

- **QUERY:** What films have a running length in the range 89-91 minutes?
- **APPROACH:** Use **BETWEEN** to specify a range of values for length.

```
SELECT film_id, title, length
FROM film
WHERE length BETWEEN 89 AND 91
ORDER BY title;
```

Results:

film_id	title	length
28	ANTHEM LUKE	91
57	BASIC EASY	90
... there are many rows that match ...		

Using LIKE for Comparisons

- The standard relational operators in a **WHERE** clause work well for numeric comparisons
- The **LIKE** allows you to perform custom string comparison
 - Partial string match using the percent sign (%) as a wildcard match

Example

```
WHERE title LIKE "Sit%"
```

- * Means the value **starts with** "Sit" (or even "sit" since SQL doesn't care about case)

```
WHERE title LIKE "%sit"
```

- * Means the value **ends with** "sit"

```
WHERE title LIKE "%sit%"
```

- * Means **contains** "sit" anywhere in the string

Example: LIKE

- **QUERY:** What films titles start with the word "Theory"?
- **APPROACH:** Use **LIKE** with a wildcard value of "Theory%" in the query.

```
SELECT film_id, title, rating
FROM film
WHERE title LIKE "Theory%";
```

Results:

film_id	title	rating
886	THEORY MERMAID	PG-13

- **QUERY:** What films have the phrase "sec" anywhere in their title?
- **APPROACH:** Use **LIKE** with a wildcard value of "%sec%" in the query.

```
SELECT film_id, title, rating
FROM film
WHERE title like "%sec%"
ORDER BY title;
```

Results:

film_id	title	rating
231	DINOSAUR SECRETARY	R
461	INSECTS STONE	NC-17
... there are many rows that match ...		

Querying for NULL Values

- **Some tables will allow NULL for some fields**
 - This does complicate queries a tiny bit
- **For example:**
 - if you are querying the film table for all films that have an `original_language_id` of 1, films that have a NULL for that columns won't be included
 - But if you are querying for all films that DO NOT have an `original_language_id` of 1, films that have a NULL for that columns ***still won't be included***.
- **You must use a specific test to match a column to NULL**

Syntax

```
WHERE column-name IS NULL
```

```
WHERE column-name IS NOT NULL
```

Example: Querying with NULL

- **QUERY:** Find all films that don't have a value for `original_language_id`

```
SELECT film_id, title
FROM film
WHERE original_language_id IS NULL;
```

- **QUERY:** Find all films that do have a value for `original_language_id`

```
SELECT film_id, title
FROM film
WHERE original_language_id IS NOT NULL;
```

SELECT DISTINCT

- The **SELECT DISTINCT** statement returns only distinct (unique) values
- When applied to a row, it returns a row the totality of all the fields is different from others returned

Syntax

```
SELECT DISTINCT column1, column2, ...  
FROM table_name;
```

- When applied to a column, it returns only unique values for the column

Syntax

```
SELECT DISTINCT(column1)  
FROM table_name;
```

Example: DISTINCT

- **QUERY:** Find all the unique prices we rent films for

```
SELECT DISTINCT(rental_rate)
FROM film;
```

Results:

```
+-----+
| rental_rate |
+-----+
| 0.99        |
| 4.99        |
| 2.99        |
+-----+
```

Comments

- You can add both single-line and multi-line comments to a SQL script
- Single line comments start with --
 - The rest of the line after the -- is ignored

Example

```
--Select all films that run 1.5 hrs or less  
SELECT film_id, title, length  
FROM film  
WHERE length <= 90;    -- 90 minutes is 1.5 hrs
```

- Multi-line comments start with /* and end with */
 - All text between /* and */ is ignored

Example

```
/*  
    Select all films that run 1.5 hrs or less  
*/  
  
SELECT film_id, title, length  
FROM film  
WHERE length <= 90;
```

Exercises- Basic Queries

Create a new folder in your `pluralsight` directory and name it `workbook-7`.

EXERCISE 1

W3Schools not only has CSS, Bootstrap, and JavaScript material, it has a detailed section on SQL. Take a few minutes to do the following practice exercises:

Go to: [w3schools.com/sql/sql_exercises](https://www.w3schools.com/sql/sql_exercises)

And Search for

1. SELECT
2. ORDER BY
3. WHERE
4. LIKE
5. BETWEEN

EXERCISE 2

Create a new project folder named `basic-queries` in the `workbook-7` directory. Create any `.sql` files directly in the `basic-queries` folder.

Create a [Github.com](https://github.com) repository named `basic-queries`.

On the empty repository screen, click the 'uploading an existing file'

Quick setup — if you've done this kind of thing before



Set up in Desktop

or

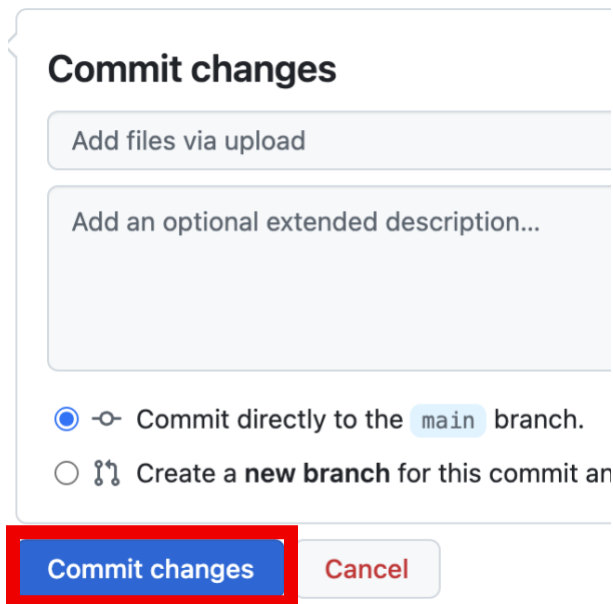
HTTPS

SSH

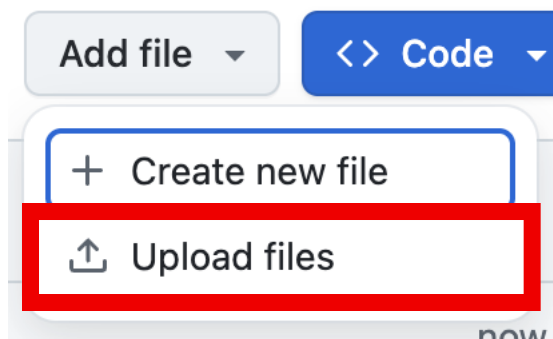
`git@github.com:topherbc/untitled.git`

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [CONTRIBUTING](#) file.

Write a useful commit message and click 'Commit changes'



After you have uploaded one or more .sql files. Click 'Add file' and then 'Upload files' and follow the same process



Use the MySQL Workbench and the Northwind database to answer the following questions

Create a new .sql file for each query and name each file with the question number, i.e. 02_product_list.sql. (This approach helps to organize and more easily find your queries later)

Alternatively, you may want to put all SQL statements in the same .sql script with comments before each query and then only run the selected query by pressing the 2nd lightning bolt.

STEPS

1. What is the name of the table that holds the items Northwind sells?
2. Write a query to list the product id, product name, and unit price of every product.
3. Write a query to list the product id, product name, and unit price of every product. Except this time, order then in ascending order by price.
4. What are the products that we carry where the unit price is \$7.50 or less?
5. What are the products that we carry where we have at least 100 units on hand? Order them in descending order by price.
6. What are the products that we carry where we have at least 100 units on hand? Order them in descending order by price. If two or more have the same price, list those in ascending order by product name.
7. What are the products that we carry where we have no units on hand, but 1 or more units of them on backorder? Order them by product name.
8. What is the name of the table that holds the types (categories) of the items Northwind sells?
9. Write a query that lists all of the columns and all the rows of the categories table? What is the category id of seafood?
10. Examine the Products table. How does it identify the type (category) of each item sold? Write a query to list all of the seafood items we carry.
11. What are the first and last names of all of the Northwind employees?
12. What employees have "manager" in their titles?
13. List the distinct job titles in employees.
14. What employees have a salary that is between \$2000 and \$2500?
15. List all the information about all of Northwind's suppliers.
16. Examine the Products table. How do you know what supplier supplies each product? Write a query to list all the items that "Tokyo Traders" supplies to Northwind

Module 3

Additional Querying Features

Section 3–1

Aggregate Functions

Aggregate Functions

- **SQL has functions that can be used to perform calculations on values from groups of rows, including**
 - The `COUNT ()` function counts the number of occurrences of a value in a specified column
 - * It can also be used to count the number of rows
 - The `SUM ()` function adds up values in the specified column
 - The `AVG ()` function returns the average value from a specific column
- **SQL also functions that can be used to find the minimum or maximum value of a column**
 - The `MIN ()` function finds the minimum value
 - The `MAX ()` function finds the maximum value

Example: COUNT ()

- **QUERY:** How many films are in the films table?
- **APPROACH:** Use the **COUNT ()** function to count the number of rows in the film table.

```
SELECT COUNT(*)  
FROM film;
```

Results:

```
+-----+  
| COUNT(*) |  
+-----+  
| 1000     |  
+-----+
```

- **QUERY:** How many distinct ratings are represented in the films table?
- **APPROACH:** Use the **COUNT ()** function combined with **DISTINCT** to count the number of ratings in the film table.

```
SELECT COUNT(DISTINCT(rating))  
FROM film;
```

Results:

```
+-----+  
| COUNT(DISTINCT(rating)) |  
+-----+  
| 5                        |  
+-----+
```

Example: SUM ()

- **QUERY:** If I wanted to watch all of the movies in the film catalog, how long would it take?
- **APPROACH:** Use the **SUM ()** function to add up all the length values in the films table.

```
SELECT SUM(length)
FROM film;
```

Results:

```
+-----+
| SUM(length) |
+-----+
| 115272      |
+-----+
```

Example: AVG ()

- **QUERY:** What is the average cost to rent a "G"-rated film?
- **APPROACH:** Use the **AVG ()** function to find the average value in the **rental_rate** column of all films whose rating is "G".

```
SELECT AVG(rental_rate)
FROM film
WHERE rating = "G";
```

Results:

```
+-----+
| AVG(rental_rate) |
+-----+
| 2.888876          |
+-----+
```

Example: MIN () and MAX ()

- **QUERY:** How short is the shortest film? What about the longest?
- **APPROACH:** Use the **MIN ()** and **MAX ()** function to examine the length.

```
SELECT MIN(length)
FROM film;
```

Results:

```
+-----+
| MIN(length) |
+-----+
| 46          |
+-----+
```

```
SELECT MAX(length)
FROM film;
```

Results:

```
+-----+
| MAX(length) |
+-----+
| 185         |
+-----+
```

Section 3–2

Grouping Results

GROUP BY clause

- The **GROUP BY** clause allows you to execute aggregate functions on "groups" of data created by the query
 - **GROUP BY** specifies the column(s) used to create the groups
 - The aggregate functions returns a value for EACH group created

Syntax

```
SELECT column1, column2, etc
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

Example: GROUP BY

- **QUERY:** How many movies are available broken down by rating (G, PG, PG-13, etc)?
- **APPROACH:** Use the **GROUP BY** clause to create groups of films by rating and then use the **COUNT ()** function to count the number of rows in each group.

```
SELECT rating, COUNT(*)  
FROM film  
GROUP BY rating;
```

Results:

rating	COUNT(*)
PG	194
G	178
NC-17	210
PG-13	223
R	195

- **QUERY:** What is the average price to rent a movie broken down by rating (G, PG, PG-13, etc)?
- **APPROACH:** Use the **GROUP BY** clause to create groups of films by rating and then use the **AVG ()** function to calculate the

average **rental_rate** of rows in each group.

```
SELECT rating, avg(rental_rate)
FROM film
GROUP BY rating;
```

RESULTS:

rating	AVG(rental_rate)
PG	3.051856
G	2.888876
NC-17	2.970952
PG-13	3.034843
R	2.9387818

Renaming Computed Fields - Using AS

- Computed fields don't have an official name in a SQL query

Example

```
SELECT rental_id, SUM(amount)
FROM sakila.payment
GROUP BY rental_id
ORDER BY rental_id;
```

	rental_id	SUM(amount)
▶	NULL	9.95
	1	2.99
	2	2.99
	3	3.99
	4	4.99
	5	5.99



- This can be a problem if you want to use it to order the results
- SQL provides the **AS** keyword to create an alias for the column name
 - You can use it for ordering or other purposes

Example

```
SELECT rental_id, SUM(amount) AS total_amount
FROM sakila.payment
GROUP BY rental_id
ORDER BY rental_id;
```

- AS can also create an alias for a table name; we will see this in the next module

Example: AS keyword

- **QUERY:** What is the average price to rent a movie broken down by rating (G, PG, PG-13, etc) and displayed in ascending order by average price?
- **APPROACH:** Use the **GROUP BY** clause to create groups of films by rating and then use the **AVG ()** function to calculate the average **rental_rate** of rows in each group. Make sure to name the value returned by the **AVG ()** function so that we can use it in the **ORDER BY** clause.

```
SELECT rating, AVG(rental_rate) AS avg_rate
FROM film
GROUP BY rating
ORDER BY avg_rating;
```

Results:

rating	avg_rate
G	2.888876
R	2.938781
NC-17	2.970952
PG-13	3.034843
PG	3.051856

HAVING clause

- The **HAVING** clause is used with the **GROUP BY** clause
- It allows you to include only those groups that meet a specified condition

Syntax

```
SELECT column1, columns2, etc  
FROM table_name  
WHERE row-condition  
GROUP BY column_name(s)  
HAVING group-condition  
ORDER BY column_name(s);
```

- The **WHERE** clause is used to select rows, the **HAVING** clause is used to select groups

Example: HAVING

- **QUERY:** What is the average rating for movies broken down by rating (G, PG, PG-13, etc)?

* NOTE: I'm not interested in the rating if there are less than 200 films in the group.

- **APPROACH:** Use the **GROUP BY** clause to create groups of films by rating and then use the **COUNT ()** function to count the number rows in each group. Only display the groups that have at least 200 rows.

```
SELECT rating, COUNT(*)
FROM film
GROUP BY rating
HAVING COUNT(*) >= 200
ORDER BY rating;
```

Results:

rating	COUNT(*)
NC-17	210
PG-13	223

Exercises – Aggregates and Grouping

EXERCISE 1

Let's look back at W3Schools again to test ourselves with aggregate functions, go to: [w3schools.com/sql/sql_exercises](https://www.w3schools.com/sql/sql_exercises) and Search for

1. Functions
2. GROUP BY

EXERCISE 2

Create a new folder in the `workbook-7` directory and call it `aggregates-and-grouping`. Add your `.sql` scripts to this folder.

Create a new remote repository named `aggregates-and-grouping`. And follow the same process of uploading your `.sql` scripts. Send your repository URL to your Instructor when you're finished.

In this exercise, will continue to execute queries against the Northwind database.

1. How many suppliers are there? Use a query!
2. What is the sum of all the employee's salaries?
3. What is the price of the cheapest item that Northwind sells?
4. What is the average price of items that Northwind sells?
5. What is the price of the most expensive item that Northwind sells?
6. What is the supplier ID of each supplier and the number of items they supply? You can answer this query by only looking at the Products table.
7. What is the category ID of each category and the average price of each item in the category? You can answer this query by only looking at the Products table.
8. For suppliers that provide at least 5 items to Northwind, what is the supplier ID of each supplier and the number of items they supply? You can answer this query by only looking at the Products table.
9. List the product id, product name, and inventory value (unit price * units on hand). Sort the results in descending order by value. If two or more have the same value, order by product name.

Section 3–3

Nested Queries

Nested Queries

- **Sometimes, you need to do a select to get one result and then use that result in another `SELECT` statement's `WHERE` clause**
 - This is often called a subquery, inner query or nested query
- **When you write subqueries, they must be enclosed within parentheses**
 - They can't have `ORDER BY` clause
- **A subquery usually has only one column in the `SELECT` clause**
 - Subqueries that return more than one row can only be used with multiple value operators such as the `IN` operator.

Syntax

```
SELECT column1, column2, etc
FROM table
WHERE column_name OPERATOR (SELECT column_name
                             FROM table1
                             WHERE condition)

GROUP BY column_name
HAVING group-condition
ORDER BY column;
```

Example: Nested Query

- **QUERY:** Which film(s) are the most expensive to replace?
- **APPROACH:** Use the SQL `max()` function to find the largest `replacement_cost` in the film table and then use that maximum cost in a different query to select the film(s) that have that replacement cost.

```
SELECT film_id, title, replacement_cost
FROM film
WHERE replacement_cost = (SELECT MAX(replacement_cost)
                          FROM film);
```

Results:

```
+-----+-----+-----+
| film_id | title                | replacement_cost |
+-----+-----+-----+
| 34      | ARABIA DOGMA         | 29.99            |
| 52      | BALLROOM MOCKINGBIRD | 29.99            |
+-----+-----+-----+
| ... there were many, many rows that matched ... |
+-----+-----+-----+
```

- **QUERY:** Which film(s) are described as documentaries and how long do they run?
- **APPROACH:** If we research the **film_text** table in the sakila database, we find it contains 3 columns named **film_id**, **title**, and **description**. We can run a query to find the films that have "documentary" in their descriptions. But the length of the film isn't available in **film_text**.

In this solution below, we keep the **film_id** values of the query that searches for documentaries and then use ANOTHER query against the film table to find all films in that 1st query's **film_id** list. Note that the where uses the keyword "in" rather than an "=" to match **film_id** values.

```
SELECT title, length
FROM film
WHERE film_id IN (SELECT film_id
                  FROM film_text
                  WHERE description LIKE "%documentary%");
```

Results:

```
+-----+-----+
| title                | length |
+-----+-----+
| AFFAIR PREJUDICE     | 117    |
| AFRICAN EGG          | 130    |
+-----+-----+
| ... many rows matched ... |
+-----+-----+
```

Exercises – Nested Queries

EXERCISE 4

In this exercise, will continue to execute queries against the Northwind database.

Create a project folder named `nested-queries` in the `workbook-7` directory and add your `.sql` scripts to this folder.

Create a new remote repository named `nested-queries`. And follow the same process of uploading your `.sql` scripts. Be sure to send your repository URL to your Instructor when you're finished.

1. What is the product name(s) of the most expensive products? HINT: Find the max price in a subquery and then use that value to find products whose price equals that value.
2. What is the order id, shipping name and shipping address of all orders shipped via "Federal Shipping"? HINT: Find the shipper id of "Federal Shipping" in a subquery and then use that value to find the orders that used that shipper.
3. What are the order ids of the orders that ordered "Sasquatch Ale"? HINT: Find the product id of "Sasquatch Ale" in a subquery and then use that value to find the matching orders from the ``order details`` table. Because the ``order details`` table has a space in its name, you will need to surround it with back ticks in the FROM clause.
4. What is the name of the employee that sold order 10266?
5. What is the name of the customer that bought order 10266?

Commit and push your queries and send your repo to your Instructor!

(OPTIONAL) EXERCISE 5

The sqlbolt.com web site has some nice tutorial and practice exercises that build up to more complex situations.

Anytime you finish an exercise early, go there and start working your way through the exercises.

Module 4

Querying Multiple Tables (JOINS)

Section 4–1

Querying Multiple Tables (JOINS)

Querying Multiple Tables (JOINS)

- Sometimes the information you want must be created by combining data in more than one table
 - In this case, SQL provides a way for us to *join* two or more tables together to get the information we want
- A join is where you take a row from one table and *join* it to a row in another table based on some condition
 - The condition is usually matching a foreign key to a primary key

employee

id	first name	last name	pay	pay grade id
100001	Greg	Smith	32000.00	1
100002	Cindy	Jones	49000.00	3
100003	Nick	Schwartz	41000.00	2
100004	Ken	McCaskill	38000.00	2

pay_grade

id	description	min val	max val
1	ES05 Pay Grade	22000.00	37000.00
2	ES10 Pay Grade	33000.00	46000.00
3	ES15 Pay Grade	39000.00	57000.00
4	ES20 Pay Grade	52000.00	75000.00

- The result set is "new" rows containing columns from all of the tables in the join
- There are several different types of JOIN operations, including:
 - INNER JOIN (most common)
 - OUTER JOIN

Inner Joins

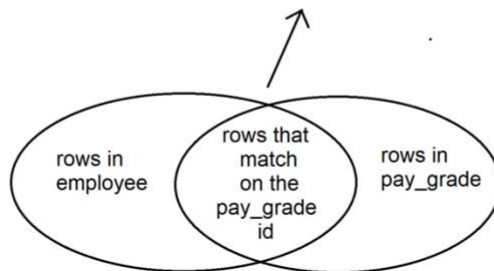
- With an **INNER JOIN**, a row in one table is joined with a row in another table based on a column match
- Only matched rows will be included in the result
 - If a row in the first table doesn't match any rows in the second table, it will be excluded
 - Similarly, if a row in the second table doesn't match a row in the first table, it will be excluded

employee

id	first name	last name	pay	pay grade id
100001	Greg	Smith	32000.00	1
100002	Cindy	Jones	49000.00	3
100003	Nick	Schwartz	41000.00	2
100004	Ken	McCaskill	38000.00	2

pay_grade

id	description	min_val	max_val
1	ES05 Pay Grade	22000.00	37000.00
2	ES10 Pay Grade	33000.00	46000.00
3	ES15 Pay Grade	39000.00	57000.00
4	ES20 Pay Grade	52000.00	75000.00



- The syntax for doing an **INNER JOIN** is a bit more complex than a simple **SELECT** because you must specify both tables and the join condition

Syntax

```
SELECT column1, columns2, ...  
FROM table1  
INNER JOIN table2  
    ON table1.column-name = table2.column-name;
```

- **You don't have to include the word `INNER`**

- A `JOIN` is considered to be an `INNER JOIN` unless specifically written as one of the other types you will learn about

Syntax

```
SELECT column1, columns2, ...  
FROM table1  
JOIN table2  
    ON table1.column-name = table2.column-name;
```

Example: INNER JOIN

- **QUERY:** We want to list each employee, along with their **pay_grade** description and the min/max salary of that pay grade.

employee

id	first name	last name	pay	pay grade id
100001	Greg	Smith	32000.00	1
100002	Cindy	Jones	49000.00	3
100003	Nick	Schwartz	41000.00	2
100004	Ken	McCaskill	38000.00	2

pay_grade

id	description	min val	max val
1	ES05 Pay Grade	22000.00	37000.00
2	ES10 Pay Grade	33000.00	46000.00
3	ES15 Pay Grade	39000.00	57000.00
4	ES20 Pay Grade	52000.00	75000.00

- **APPROACH:** Join the employee table to the **pay_grade** table and match **employee.pay_grade_id** to **pay_grade.id**

```
SELECT employee.id, first_name, last_name, description, min_val, max_val
FROM employee
JOIN pay_grade
  ON employee.pay_grade_id = pay_grade.id;
```

Results:

id	first_name	last_name	description	min_val	max_val
100001	Greg	Smith	ES05 Pay Grade	22000.00	37000.00
100002	Cindy	Jones	ES15 Pay Grade	39000.00	46000.00
100003	Nick	Schwartz	ES10 Pay Grade	33000.00	46000.00
100004	Ken	McCaskill	ES10 Pay Grade	33000.00	46000.00

We could have added the `WHERE`, `ORDER BY`, `GROUP BY`, and `HAVING` clauses if we wanted. And although this example does a `JOIN` on two tables, you can `JOIN` as many tables as you need to continue to add additional `JOIN` clauses.

Beyond Inner Joins

- The **INNER JOIN** puts data together from multiple tables based on common values in the tables.
 - A row is **ONLY** created in the result set when a row in one table and a row in the other table match a specified condition
- But this may not give us what we need
- For example, consider a commercial business
 - A company take orders from customers
 - When an order is created for a customer, the customer id is stored in the order to identify which customer the order is for.
 - However, the company also allows cash orders and doesn't have a customer id for those orders
 - * This means the order table must allow NULLs
- If we try to join the order table to the customer table, these orders would be excluded

order

id	sold_date	customer_id
1	2021-05-21 10:02:00	104
2	2021-05-21 11:13:45	102
3	2021-05-21 12:06:13	NULL
4	2021-05-22 10:00:00	103
5	2021-05-23 11:02:34	NULL
6	2021-05-25 11:39:40	103

customer

id	name	email
101	Ezra Aiden	theater_guy@gmail.com
102	Ian Auston	gamer05@yahoo.com
103	Siddalee Grace	susa@gmail.com
104	Elisha Aslan	gamer06@yahoo.com

An INNER JOIN would exclude cash customers in order where the customer_id is NULL

```
SELECT order.id, sold_date, name, email
FROM `order`
JOIN customer
  ON order.customer_id = customer.id;
```

Results:

id	sold_date	name	email
1	2021-05-21 10:02:00	Elisha Aslan	gamer06@yahoo.com
2	2021-05-21 11:13:45	Ian Auston	gamer05@yahoo.com
4	2021-05-22 10:00:00	Siddalee Grace	susa@gmail.com
6	2021-05-25 11:39:40	Siddalee Grace	susa@gmail.com

- Orders where the `customer_id` is `NULL` do NOT show, but we wanted all orders!

Outer Joins

- **An OUTER JOIN is used when might want rows in one table that don't match rows in the other table**
 - You must specify which of the two tables isn't required to have matching data in its JOIN column
- **You do this by creating a LEFT OUTER JOIN or a RIGHT OUTER JOIN**
 - The first table listed in the JOIN is considered to be on the left and the second is considered to be on the right
- **For a LEFT OUTER JOIN, the first table doesn't require matching data to be included**
 - Selected columns without a matching value will be NULL
- **For a RIGHT OUTER JOIN, the second table doesn't require matching data to be included**
 - Selected columns without a matching value will be NULL
- **NOTE: the word OUTER isn't actually required but is often included for readability**

Example: Outer Joins

order

id	sold_date	customer_id
1	2021-05-21 10:02:00	104
2	2021-05-21 11:13:45	102
3	2021-05-21 12:06:13	NULL
4	2021-05-22 10:00:00	103
5	2021-05-23 11:02:34	NULL
6	2021-05-25 11:39:40	103

customer

id	name	email
101	Ezra Aiden	theater_guy@gmail.com
102	Ian Auston	gamer05@yahoo.com
103	Siddalee Grace	susa@gmail.com
104	Elisha Aslan	gamer06@yahoo.com

orders 3 and 5 are included
in a LEFT OUTER JOIN

- **QUERY:** What orders were sold when?

```
SELECT order.id, sold_date, name, email
FROM order
LEFT JOIN customer
    ON order.customer_id = customer.id;
```

Results:

id	sold_date	name	email
1	2021-05-21 10:02:00	Elisha Aslan	gamer06@yahoo.com
2	2021-05-21 11:13:45	Ian Auston	gamer05@yahoo.com
3	2021-05-21 12:06:13	NULL	NULL
4	2021-05-22 10:00:00	Siddalee Grace	susa@gmail.com
5	2021-05-23 11:02:34	NULL	NULL
6	2021-05-25 11:39:40	Siddalee Grace	susa@gmail.com

- **When there are many tables involved and your queries are complex, it can be helpful to have a visual representation of the database**

[illegible]

- 85

Exercises - Joins

EXERCISE 1

Let's start by checking out the exercises at W3Schools. Go to: [w3schools.com/sql/sql_exercises](https://www.w3schools.com/sql/sql_exercises) and Search for

1. JOIN

EXERCISE 2

Now take a few minutes to look at this great visual diagram of the different types of joins: <https://datalemur.com/blog/sql-joins-infographics>

EXERCISE 3

Create a new folder called `joins` to put your `.sql` files for this exercise.

Create a new remote repository named `joins`. And follow the same process of uploading your `.sql` scripts. Be sure to send your repository URL to your Instructor when you're finished.

Let's continue working with Northwind.

1. List the product id, product name, unit price and category name of all products. Order by category name and within that, by product name.
2. List the product id, product name, unit price and supplier name of all products that cost more than \$75. Order by product name.
3. List the product id, product name, unit price, category name, and supplier name of every product. Order by product name.
4. What is the product name(s) and categories of the most expensive products?
HINT: Find the max price in a subquery and then use that in your more complex query that joins products with categories.
5. List the order id, ship name, ship address, and shipping company name of every order that shipped to Germany.
6. List the order id, order date, ship name, ship address of all orders that ordered "Sasquatch Ale"?

Module 5

Modifying the Data and the Database

Section 5–1

Inserting, Updating and Deleting Data

Inserting, Updating and Deleting Data

- Up until now, most of the SQL we have shown has been used to query data
- However, you often need to make changes to the data in a database
- You do this using the SQL statements **INSERT**, **UPDATE**, and **DELETE**

INSERT INTO Statement

- If you want to add new rows to a table, you use the **INSERT INTO** statement.
- There are two possible techniques:
 - In the first, you specify the table name and the values of the rows to be added
 - In the second, you specify the table's column names and the values of the rows to be added
 - * In this version, you could omit some columns and they would become `NULL`

Syntax

```
INSERT INTO table-name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

Syntax

```
INSERT INTO table-name
VALUES (value1, value2, value3, ...);
```

- If you attempt to add a record with `NULL` in a non-nullable column, the **INSERT** statement will fail

Example: INSERT INTO

- **TASK:** Add a new country to the sakila country table

- **STATEMENT (option 1):**

```
INSERT INTO country(country_id, country, last_update)
VALUES(110, "Zimbabwe", NOW());
```

- **STATEMENT (option 2):**

```
INSERT INTO country
VALUES(110, "Zimbabwe", NOW());
```

UPDATE Statement

- **Use the UPDATE statement to edit rows in a table**
 - It allows you to change the value in one or more columns of a row
- **Use the WHERE clause to specify which row(s) will have the changes made to them**

Syntax

```
UPDATE table-name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

- ***Be careful with the UPDATE statement. If you forget the WHERE clause, all of the records in that table will be updated!***
- **MySQL workbench will generate an error if the update condition isn't based on a primary key**
 - You can turn that off with the following code
 - Turn the safety setting back on again after your query

Example

```
SET SQL_SAFE_UPDATES=0;  
-- run the update w/o PK  
SET SQL_SAFE_UPDATES=1;
```

Example: UPDATE

- **TASK:** Change the first and last name for the customer whose **customer_id** is 2.

```
UPDATE customer
SET first_name = 'PATTY', last_name = 'JOHNSTON'
WHERE customer_id = 2;
```

- **TASK:** Change all PATTY first names to PATRICE.

```
SET SQL_SAFE_UPDATES=0;

UPDATE customer
SET first_name = 'PATRICE'
WHERE first_name = 'PATTY';

SET SQL_SAFE_UPDATES=1;
```

DELETE Statement

- To delete one or more rows from a table, use the **DELETE** statement

Syntax

```
DELETE FROM table-name  
WHERE condition;
```

- ***Be careful with the DELETE statement. If you forget the WHERE clause, all of the records in that table will be updated!***
- **Deleting a record is a precarious action because the row you want to delete often is referenced in many tables!**
 - For example, if you try to delete a specific film from the film table, you will now have a reference in inventory to a `film_id` that doesn't exist
- **NOTE: Many companies don't delete records... Instead they have a column whose name might be "active" or something similar**
 - You set active to true when the row is created and mark it "deleted" by using the `UPDATE` statement to set active to false.
 - This will allow the database to maintain referential integrity between related tables.

Example: DELETE

- **TASK:** Delete all references to the payment whose **payment_id** is 100

```
DELETE FROM payment  
WHERE payment_id = 100;
```

Exercises – Insert, Update and Delete

EXERCISE 1

Let's start by checking out the exercises at W3Schools.

Go to: [w3schools.com/sql/sql_exercises](https://www.w3schools.com/sql/sql_exercises)

And Search for

1. INSERT
2. UPDATE
3. DELETE

EXERCISE 2

Create a new folder called `insert-update-delete` to hold your `.sql` scripts.

Create a new remote repository named `insert-update-delete`. And follow the same process of uploading your `.sql` scripts. Be sure to send your repository URL to your Instructor when you're finished.

Let's continue working with Northwind.

1. Add a new supplier.
2. Add a new product provided by that supplier
3. List all products and their suppliers.
4. Raise the price of your new product by 15%.
5. List the products and prices of all products from that supplier.
6. Delete the new product.
7. Delete the new supplier.
8. List all products.
9. List all suppliers.

Section 5–2

Creating a Table

Creating a New Table

- You can create tables (and even databases) using SQL
- The **CREATE TABLE** statement is used to create a table

Syntax

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    ....  
);
```

- A list of MySQL data types can be found here:

<https://dev.mysql.com/doc/refman/8.0/en/data-types.html>

- You can mark a field as a primary key using **PRIMARY_KEY**

Syntax

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    ....  
    PRIMARY KEY (column1)  
);
```

Creating a New Table

- You can mark fields as not being nullable

Syntax

```
CREATE TABLE table_name (  
    column1 datatype NOT NULL,  
    column2 datatype,  
    column3 datatype NOT NULL,  
    ....  
    PRIMARY KEY (column1)  
);
```

- You can also mark fields as being an auto-increment field
 - This happens frequently when you don't want the user to be responsible for generating unique primary keys
 - * MySQL uses the `AUTO_INCREMENT` keyword to specify an auto-increment field
 - * It must be associated with an integer datatype

Syntax

```
CREATE TABLE table_name (  
    column1 INT NOT NULL AUTO_INCREMENT,  
    column2 datatype,  
    column3 datatype NOT NULL,  
    ....  
    PRIMARY KEY (column1)  
);
```

Example: CREATE TABLE

- **TASK:** Create a new table to track advertised sales

```
CREATE TABLE advertisements (  
    AdId int NOT NULL,  
    Title varchar(50) NOT NULL,  
    MagicCode varchar(9),  
    PercentOff float NOT NULL,  
    PRIMARY KEY(AdId)  
);
```

- **TASK:** Create a new table to track advertised sales but use an auto-increment key

```
CREATE TABLE advertisements (  
    AdId int NOT NULL AUTO_INCREMENT,  
    Title varchar(50) NOT NULL,  
    MagicCode varchar(9),  
    PercentOff float NOT NULL,  
    PRIMARY KEY(AdId)  
);
```

Module 6

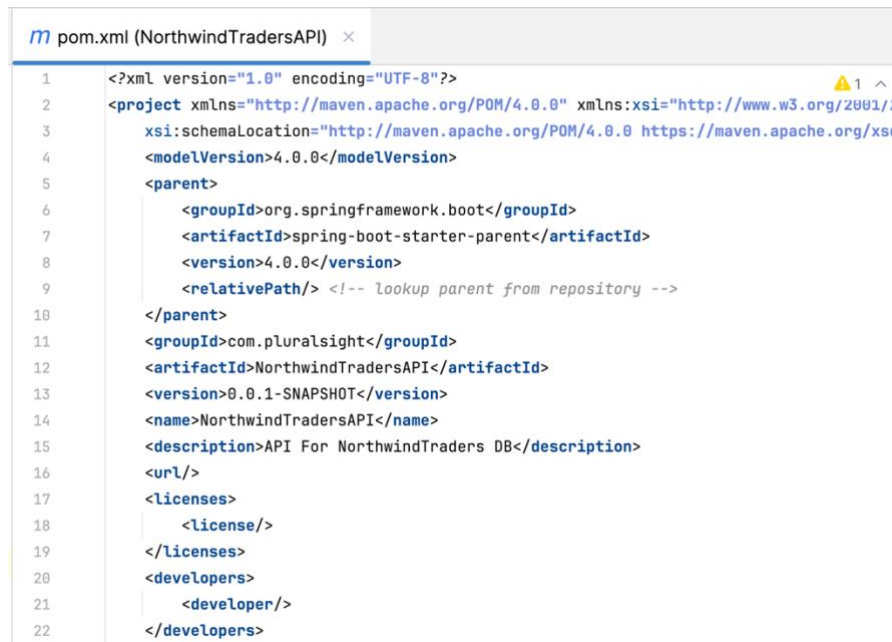
Connecting SQL to Java

Section 6–1

Maven Dependencies

The pom.xml file

- The `pom.xml` file is short for Project Object Model
- This is XML file containing metadata and configuration for building your project



```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/
3   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsi
4   <modelVersion>4.0.0</modelVersion>
5   <parent>
6     <groupId>org.springframework.boot</groupId>
7     <artifactId>spring-boot-starter-parent</artifactId>
8     <version>4.0.0</version>
9     <relativePath/> <!-- lookup parent from repository -->
10  </parent>
11  <groupId>com.pluralsight</groupId>
12  <artifactId>NorthwindTradersAPI</artifactId>
13  <version>0.0.1-SNAPSHOT</version>
14  <name>NorthwindTradersAPI</name>
15  <description>API For NorthwindTraders DB</description>
16  <url/>
17  <licenses>
18    <license/>
19  </licenses>
20  <developers>
21    <developer/>
22  </developers>
```

- Common modifications include
 - Changing the target Java version
 - Adding or updating project dependencies
- When you modify `pom.xml`, reload Maven dependencies
 - This should appear on the right side of IntelliJ when you modify the `pom.xml`



Adding the MySQL Driver

- **JDBC requires a database-specific driver to establish a connection**
 - The driver is an external library that implements the communication protocol for your specific database
 - For MySQL, we add the `mysql-connector-java` dependency
- **Add this dependency to your `pom.xml`:**

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
</dependency>
```

- **After adding:**
 - Reload Maven dependencies
 - Expand External Libraries in the project panel to confirm the jar was imported
 - Import statements for JDBC classes will now resolve

Adding the DataSource Dependency

- **Modern JDBC applications use a DataSource for managing database connections**
 - This is more efficient than the older DriverManager approach
 - Handles connection pooling automatically
- **Add this dependency to your pom.xml:**

```
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-dbcp2</artifactId>
  <version>2.12.0</version>
</dependency>
```

Section 6–2

Introduction to JDBC

Java Applications and Databases

- **Java applications allow users to interact with computers**
 - In most cases they allow users to interact with data stored in databases
 - To connect a Java application to a database, we need a bridge to the JDBC

```
Enter a number (or "done"): 15
Enter a number (or "done"): 23
Enter a number (or "done"): -19
Enter a number (or "done"): 84
Enter a number (or "done"): 5
Enter a number (or "done"): done
-----
Total: 108
```

- **Databases store information like customer records, products, and orders**

	ID	Name	CountryCode	District	Population
▶	1	Kabul	AFG	Kabul	1780000
	2	Qandahar	AFG	Qandahar	237500
	3	Herat	AFG	Herat	186800
	4	Mazar-e-Sharif	AFG	Balkh	127800
	5	Amsterdam	NLD	Noord-Holland	731200
	6	Rotterdam	NLD	Zuid-Holland	593321
	7	Haag	NLD	Zuid-Holland	440900
	8	Utrecht	NLD	Utrecht	234323
	9	Eindhoven	NLD	Noord-Brabant	201843
	10	Tilburg	NLD	Noord-Brabant	193238
	11	Groningen	NLD	Groningen	172701
	12	Breda	NLD	Noord-Brabant	160398

- **Customer information, products, orders, etc.**
 - The most useful applications connect users to that data through a well-defined interface
- **The most useful applications connect users to data**

What is JDBC

- **JDBC (Java Database Connectivity) is a specification and API that lets Java programs access database management systems**
 - The API provides interfaces and classes for:
 - * Connecting to databases
 - * Executing queries
 - * Processing results
- **JDBC is database agnostic**
 - This means the same Java code works with MySQL, PostgreSQL, Oracle, SQL Server
 - Only the driver and connection URL need to change when switching databases

JDBC Architecture

- There are a few key components of the JDBC
 - The **Driver** Implements communication protocol for a specific database engine
 - The **Connection** represents the active link between your Java application and the database
 - The **Statement** and **PreparedStatement** executes SQL queries
 - * The **PreparedStatement** executes queries safely
 - The **ResultSet** contains query results

JDBC Architecture



The Database URL

- The connection string similar to a website url, this is the location for the database connection

Format

```
jdbc:mysql://[host][:port]/[database]
```

//Examples

```
jdbc:mysql://localhost:3306/northwind
```

```
jdbc:mysql://localhost:3306/sakila
```

- Components of the url:
 - * `jdbc` is the protocol prefix, required for all JDBC URLs
 - * `mysql` is the database engine
 - * `localhost:3306` is the server address and port number
 - * `northwind` is the database name to connect to

Creating and Connection to a DataSource

- **A DataSource manages database connections on behalf of your application**
 - This is the modern approach, replacing the older DriverManager pattern
 - It handles connection pooling behind the scenes, which helps with improving performance
 - Implements the `javax.sql.DataSource` interface

Example

```
import org.apache.commons.dbcp2.BasicDataSource;

BasicDataSource dataSource = new BasicDataSource();

dataSource.setUrl("jdbc:mysql://localhost:3306/northwind");

dataSource.setUsername(username);

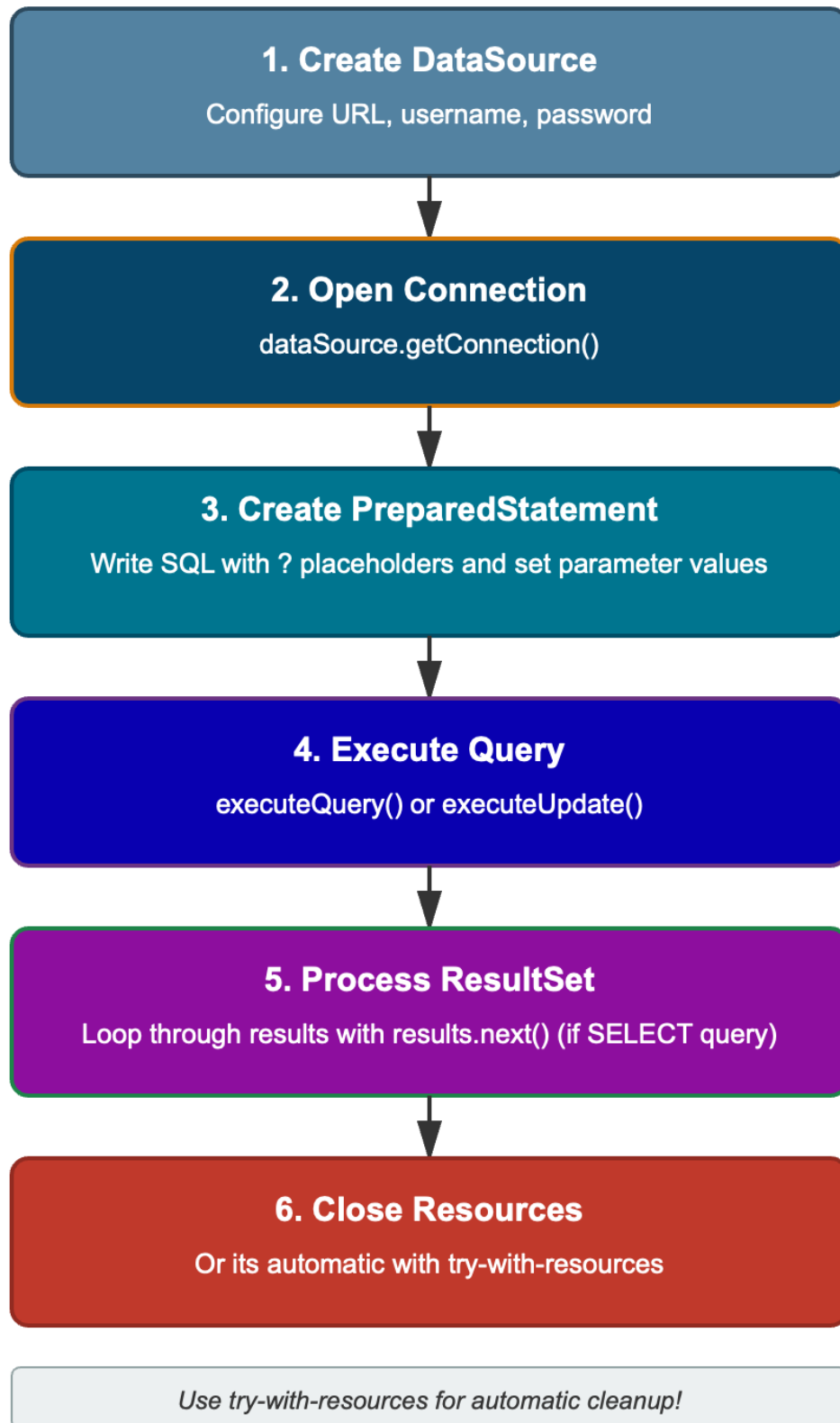
dataSource.setPassword(password);
```

- Once your `DataSource` is configured, you request a connection from it:

```
Connection connection = dataSource.getConnection();
```

- **Important:** Connections are expensive resources and should always be closed when finished using them

The JDBC Programming Process



Try-With-Resources

- **Java's try-with-resources syntax handles resource cleanup automatically**
 - Any resource declared inside the try parentheses is closed automatically when the block exits
 - This eliminates the need for explicit `close()` calls or finally blocks
 - Any object implementing `AutoCloseable` can be used
 - * `Connection`, `PreparedStatement`, and `ResultSet` all implement `AutoCloseable`

Syntax

```
try (Connection conn = dataSource.getConnection();
    PreparedStatement stmt = conn.prepareStatement(sql)) {
    // Use connection and statement here
    // They will close automatically at end of the code block
} catch (SQLException e) {
    e.printStackTrace();
}
```

- **Using this new syntax has a few benefits**
 - Cleaner code and more readable code
 - Guaranteed resource cleanup, even if an exception is thrown
 - Handles exceptions that occur during the closing of resources

Example

```
import org.apache.commons.dbcp2.BasicDataSource;
import java.sql.*;

public class DatabaseExample {
    public static void main(String[] args) {
        // Setup DataSource
        BasicDataSource dataSource = new BasicDataSource();
        dataSource.setUrl("jdbc:mysql://localhost:3306/northwind");
        dataSource.setUsername("root");
        dataSource.setPassword("password");

        // Query the database
        String sql = "SELECT ProductName, UnitPrice FROM products";

        try (Connection conn = dataSource.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                String name = rs.getString("ProductName");
                double price = rs.getDouble("UnitPrice");
                System.out.printf("%s: $%.2f\n", name, price);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Processing the ResultSet

- **The `ResultSet` contains the rows returned by a `SELECT` query**
 - Acts like a cursor starting positioned before the first row
 - Each call to `next()` advances the cursor and returns true if a row exists
- **You can retrieve the data by using type specific getter methods**
 - By column name: `getString("ProductName")`
 - By column index: `getString(1)` (1-based indexing!)
 - Type-specific methods: `getInt()`, `getDouble()`, `getDate()`, `getBoolean()`, etc.

Example

```
ResultSet rs = stmt.executeQuery();

while (rs.next()) {
    // By column name (preferred)
    String name = rs.getString("ProductName");
    double price = rs.getDouble("UnitPrice");

    // Or by index (1-based)
    String name2 = rs.getString(1);
    double price2 = rs.getDouble(2);
}
```

Exercise – NorthwindTraders Pt. 1

EXERCISE 1

In this exercise you will create a Maven project and make your first JDBC connection to the Northwind database. Complete your work in the workbook-6 folder.

1. Create a new project named `northwind-traders` with GroupId `com.pluralsight`
2. In your `pom.xml`, add each dependency for the MySQL Driver and Commons DBCP2 in a `<dependencies>` tag

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
</dependency>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
</dependency>
```

3. Connect to the Northwind database at `jdbc:mysql://localhost:3306/northwind`
4. Execute a query to select all products and display each product name to the screen
5. **Commit and push your code**

Section 6–3

Working with Prepared Statements

What is a PreparedStatement?

- **A PreparedStatement is a parameterized SQL query that is compiled once and executed safely**
- **PreparedStatement and Statement are different**
 - Statements are used for basic SQL execution
 - PreparedStatement are used with parameterized queries with safety features, this is preferred
- **Some of the key advantages are**
 - Preventing SQL injection attacks
 - Supporting parameterized queries with ? placeholders
 - Providing better performance for repeated queries through a process of pre-compilation
 - Handling automatic type conversion for Java values
- **Most SQL queries require some form of user input**
 - Dynamically building a query through String concatenation is not considered safe
 - * We will discuss SQL Injection attacks shortly

Creating a PreparedStatement

- For a query with no parameters, pass the SQL string directly

```
String sql = "SELECT first_name, last_name FROM customer";
PreparedStatement stmt = connection.prepareStatement(sql);
```

- For a query that accepts user input, use `?` as a placeholder for each parameter

```
String first_name = "Hector";
String sql = "SELECT first_name, last_name FROM customer WHERE last_name LIKE ?";
PreparedStatement stmt = connection.prepareStatement(sql);

stmt.setString(1, "%" + last_name + "%");
// The 1 here is the index which refers to the first ? in the query
```

- Compare this with injecting a variable into a statement using concatenation

```
String last_name = "Hector";
String sql = "SELECT first_name, last_name FROM customer WHERE last_name LIKE %" +
    lastName;

Statement statement = connection.createStatement();
```

Setting Parameter Values

- Use setter methods to assign a value to each ? placeholder before executing
- Some of the available setter methods are
 - `setString(index, value)`
 - `setInt(index, value)`
 - `setDouble(index, value)`
 - `setDate(index, value)`
 - `setBoolean(index, value)`
- **Important:** Parameters use 1-based indexing. The first ? in your SQL is index 1, not 0.

```
String sql = "SELECT * FROM products WHERE CategoryID = ? AND UnitPrice > ?";

PreparedStatement stmt = conn.prepareStatement(sql);
stmt.setInt(1, categoryId);    // First ? is CategoryID
stmt.setDouble(2, minPrice);  // Second ? is UnitPrice
```

Executing Queries

- **PreparedStatement** provides three methods to execute queries with

- `executeQuery()` is used primarily for `SELECT` statements

```
ResultSet rs = stmt.executeQuery();  
// Returns a ResultSet containing the selected rows
```

- `executeUpdate()` is used for `INSERT`, `UPDATE`, and `DELETE` statements

```
int rowsAffected = stmt.executeUpdate();  
// Returns the number of rows that were changed
```

- `execute()` is used for any statement type of statement, including `DDL`

```
boolean hasResults = stmt.execute();  
// Returns true if a ResultSet is available, false otherwise
```

Example

```
public void searchCustomers(String lastNamePrefix) {

    String sql = "SELECT first_name, last_name, city " +
        "FROM customer " +
        "WHERE last_name LIKE ? " +
        "ORDER BY first_name";

    try (Connection conn = dataSource.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {

        stmt.setString(1, lastNamePrefix + "%");

        try (ResultSet rs = stmt.executeQuery()) {
            while (rs.next()) {
                System.out.printf("%s %s - %s\n",
                    rs.getString("first_name"),
                    rs.getString("last_name"),
                    rs.getString("city"));
            }
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

Exercise – NorthwindTraders Pt. 2

Continue working on the NorthwindTraders project. Modify your product query to display additional fields in a formatted layout.

EXERCISE 2

1. Modify your query to display all products with: Product ID, Product Name, Unit Price, Units in Stock
2. Choose a display format that is easy to read. Two options are shown below

Option 1: Stacked

```
Product Id: 1
Name:      Chai
Price:     18.00
Stock:     39
-----
```

Option 2: Tabular

Id	Name	Price	Stock
----	-----	-----	-----
1	Chai	18.00	39
2	Chang	19.00	17

3. Commit and push your code
4. Share your repo with your Instructor

Exercise – NorthwindTraders Pt. 3

Continue with NorthwindTraders. Build a home screen menu that lets the user choose what data to display.

EXERCISE 3

1. Create a home screen menu

```
What do you want to do?  
  1) Display all products  
  2) Display all customers  
  0) Exit  
Select an option:
```

2. Add logic to query the Customers table when option 2 is selected

3. Display the Contact Name, Company Name, City, Country, Phone ordered by country

4. Use `try-with-resources` for automatic resource management

5. Commit and push your code

6. Share your repo with your Instructor

Section 6–4

SQL Injection Prevention

What is SQL Injection?

- **SQL injection is a security vulnerability where an attacker manipulates a SQL query**
- **It typically happens when a user's input is concatenated into a String**
- **The consequences can include exposing sensitive data, modifying records, or destroying entire tables**
- **Warning, never do this!**

```
// DANGEROUS – never concatenate user input into SQL
String userInput = /* value from user */;
String sql = "SELECT * FROM users WHERE username = '" + userInput + "'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```



How SQL Injection Works

- There are a few different types of SQL injection attacks
- ***Authentication Bypass*** can attempt to bypass security measures
 - Imagine a login form where a username is passed directly into a query

```
// User enters: admin' OR '1'='1
String sql = "SELECT * FROM users WHERE username = '" + username + "'";
```

- The resulting query will look like this

```
SELECT * FROM users WHERE username = 'admin' OR '1'='1'
```

- Because '1'='1' is always true, this query returns every row
- This causes authentication to be bypassed entirely

- ***Data Destruction*** is just as dangerous

- A more destructive attack uses input like: Robert'); DROP TABLE products;--

```
// User enters: Robert'); DROP TABLE products;--
String sql = "SELECT * FROM users WHERE username = '" + username + "'";
```

- This executes two statements — the second permanently destroys your products table:

```
SELECT * FROM users WHERE username = 'Robert');
DROP TABLE products;--
```

Why PreparedStatement are Safer

- **PreparedStatement treats every parameter value as data**
 - This means the malicious query is executed specifically using the set data type, not as executable SQL
- **When a user attempts injection, the malicious input will be escaped automatically**

```
// SAFE – the ? placeholder isolates the value from the SQL structure
String sql = "SELECT * FROM users WHERE username = ?";
PreparedStatement stmt = conn.prepareStatement(sql);
stmt.setString(1, userInput);
```

- If the user enters `admin' OR '1'='1`, the database looks for that exact string as the username
- **When no SQL characters are interpreted, there are no rows to match, and the attack will fail**

Exercise – NorthwindTraders Pt. 4

Continue with NorthwindTraders. Practice safe user input handling with `PreparedStatement` by adding a category-filtered product search.

EXERCISE 4

1. Add a new menu option: 3) Display all categories
2. Query the Categories table and display Category ID and Category Name, ordered by category ID
3. Prompt the user to enter a category ID, then query Products for all products in that category
4. Display the Product ID, Product Name, Unit Price, and Units in Stock
5. Test with the following inputs to verify safe handling

```
Normal input:      1
Injection attempt: 1 OR 1=1
Injection attempt: 1; DROP TABLE products;--
```

6. Your `PreparedStatement` should handle all inputs without error or data exposure
7. **Commit and push your code to GitHub**
8. **Share your repo with your Instructor**

Module 7

Continued Learning

Other Features

- **There are many SQL and MySQL features that we aren't discussing, including:**
 - dropping tables
 - defining constraints
 - creating and using views
 - creating and using stored procedures
 - working with triggers
- **It never hurts a developer to learn more SQL!**

Other SQL Learning Tools

- There are many applications to help increase your SQL capabilities!
 - sqlzoo.net
 - codewars.com
 - sqlbolt.com

Section 7–1

Practice Repository

Practice Problems

- We have a repository with additional practice problems and projects
- Your Instructor will inform you which exercises and projects you will work on within this repository.
- If the files contain -optional, you are more than welcome to work on these as you please.
- github.com/Pluralsight-ILT/YUU-Java-Academy-Bonus

Using AI Tools for SQL Data Generation

- **Let's look at how AI tools like Claude and ChatGPT can help with some of the repetitive tasks you'll encounter when working with MySQL later in this course.**
- **Automating Tedious Database Tasks**
 - When working with databases, you'll often need sample data to test your queries and applications
 - Creating this data manually can be extremely time-consuming
 - AI tools can help generate test data while you focus on learning database concepts
- **Appropriate Uses for Database Work**
 - **Generate `INSERT` statements for test data**
 - * Instead of writing dozens of similar statements, describe the pattern and let AI generate them
 - * **Example:** "I need 10 `INSERT` statements for a student's table with random names, IDs, and GPAs"
 - **Generate `INSERT` statements for test data**
 - * Create sample schemas with constraints
 - * Describe the entities and relationships you need, and get DDL statements to create them
 - * Always review the generated SQL to understand how constraints and relationships are established

- **Format and clean up existing SQL**

- If you have working but messy SQL, AI can help make it more readable

- **Example: Generating Test Data**

I'm creating a books database for my Java application. Can you generate 5 INSERT statements for a 'books' table with columns for id (INT), title (VARCHAR), author (VARCHAR), publication_year (INT), and price (DECIMAL)?"

Reasoned about SQL INSERT statements for a few seconds >

Below are five sample SQL INSERT statements for a `books` table with the columns `id`, `title`, `author`, `publication_year`, and `price`:

```
sql Copy

INSERT INTO books (id, title, author, publication_year, price)
VALUES (1, 'The Great Gatsby', 'F. Scott Fitzgerald', 1925, 10.99);

INSERT INTO books (id, title, author, publication_year, price)
VALUES (2, '1984', 'George Orwell', 1949, 9.99);

INSERT INTO books (id, title, author, publication_year, price)
VALUES (3, 'To Kill a Mockingbird', 'Harper Lee', 1960, 12.99);

INSERT INTO books (id, title, author, publication_year, price)
VALUES (4, 'Pride and Prejudice', 'Jane Austen', 1813, 8.99);

INSERT INTO books (id, title, author, publication_year, price)
VALUES (5, 'The Catcher in the Rye', 'J.D. Salinger', 1951, 11.50);
```

These statements assume that the `books` table already exists in your database with the specified columns and appropriate data types. Adjust the values as needed for your application.

📄 👍 🗨️ 🔊 ↺

- **Important Guidelines:**

- **Always understand the SQL you're using**
- Don't just copy-paste generated SQL without reviewing it
- Make sure you understand how each statement works

- **Focus on learning database principles**

- Using AI to generate test data is fine
- Using AI to solve your database design assignments is not

- **Keep your data relevant**

- When asking for sample data, specify realistic constraints
- Example: "Generate customer data with ages between 18-80"

- **Remember**

- The goal is to save time on mechanical tasks, not to avoid learning
- As you gain experience, you'll likely write scripts to generate test data
- Using AI for test data generation is similar to using other tools like data generators or scripts
- Your instructor's guidelines always take precedence, if in doubt, ask!